



UNIVERSIDAD DEL VALLE

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

Manual Técnico – TOTEM

ESTUDIANTES: Alex Bryam Barrera Alarcon
Rommel Rodrigo Gutierrez Herrera
Mauricio Waly Linaja Canty
Edson Marcelo Cayo Ali

MATERIA: Proyecto de Sistemas II

GRUPO: B

DOCENTE: Ing. Gaston Silva Sanchez

Cochabamba - Bolivia

Gestión 1 – 2026

Índice

Contenido

1. Integrantes – Roles.....	1
2. Introducción:.....	1
3. Descripción / objetivo del proyecto:	2
4. Link al Video ilustrativo en YouTube	2
5. Listado de los Requisitos Funcionales del Sistema	2
RF-01. Autenticación y gestión de sesión	2
RF-02. Control de acceso por roles y campus.....	3
RF-03. Gestión de usuarios.....	3
RF-04. Invitación de usuarios	4
RF-05. Cambio y recuperación de contraseña	4
RF-06. Gestión de tótems	5
RF-07. Gestión de contenidos	5
RF-08. Gestión de preguntas y respuestas en contenidos PDF.....	6
RF-09. Gestión de asignaciones	7
RF-10. Gestión de notificaciones.....	8
RF-11. Dashboard administrativo	9
RF-12. Visualización en el cliente tótem	9
RF-13. Interacción mediante cámara y gestos	10
RF-14. Preguntas por voz	11
RF-15. Retorno automático al modo normal	11
6. Arquitectura del software	11
6.1 Diagrama de arquitectura.....	12
6.2 Estructura general del sistema	12
6.3 Componente principales.....	13
6.4 Interacción entre componentes.....	14
6.5 Patrones y organización utilizados	15
7. Base de datos.....	16
7.1 Diagrama completo y actual.....	16

7.2 Organización de scripts en el repositorio Git	17
7.3 Tablas principales de la base de datos	17
7.4 Script simple.....	18
8. Listado de Roles más sus credenciales de todos los Admin / Users del sistema	20
8.1 Acceso al sistema desplegado	20
8.2 Roles del sistema	20
8.3 Credenciales del entorno desplegado	20
8.4 Credenciales de prueba del archivo 02_seed.sql.....	21
8.5 Acceso al cliente tótem.....	21
8.6 Enlace y recursos del proyecto	21
9. Requisitos del sistema	22
9.1 Requerimientos de Hardware – Cliente	22
9.2 Requerimientos de Software – Cliente.....	23
9.3 Requerimientos de Hardware – Servidor / Hosting / Base de Datos	23
9.4 Requerimientos de Software – Servidor / Hosting / Base de Datos	24
9.5 Requerimientos del Proyecto.....	25
9.6 Variables de Entorno Necesarias	25
9.7 Requisitos para Ejecución de Docker	25
10. Instalación y configuración	26
10.1 Requisitos previos	26
10.2 Estructura general del proyecto	27
10.3 Instalación local	27
10.4 Instalación con Docker Compose	30
10.5 Configuración de servicios externos	31
10.6 Configuración del sistema desplegado	32
10.7 Credenciales de acceso	33
11. Procedimiento de hosteado / Hosting.....	34
11.1 Distribución del sistema en hosting	34
11.2 Sitio web en Vercel.....	35
11.3 API / Backend en Render	36
11.4 Base de datos en Render PostgreSQL	37
11.5 Almacenamiento de archivos en Supabase Storage.....	38

11.6 Correos con SMTP Brevo	39
11.7 API de clima Open-Meteo	39
11.8 Credenciales de acceso	40
12. GIT.....	40
12.1 Rama final de entrega	41
12.2 Ramas utilizadas durante el desarrollo	41
12.3 Flujo de trabajo utilizado	41
12.4 Entrega del proyecto	42
12.5 Repositorio del proyecto	42
13. Dockerizado Del Backend, Frontend y Base de Datos.....	43
13.1 Componentes dockerizados	43
13.2 Proceso de dockerizado	43
13.3 Configuración principal.....	45
13.4 Como hacer correr el sistema.....	45
13.5 Inicialización de la base de datos.....	46
13.6 Acceso a los servicios	47
13.7 Credenciales de base de datos en Docker	48
13.8 Credenciales de aplicación	48
13.9 Persistencia de datos	48
14. Personalización y configuración	49
14.1 Archivos principales de configuración	49
14.2 Configuración general del sistema	50
14.3 Configuración de base de datos	50
14.4 Configuración de seguridad y sesiones.....	51
14.5 Configuración de correo SMTP.....	52
14.6 Configuración de almacenamiento de archivos.....	52
14.7 Configuración del cliente tótem	53
14.8 Configuración del modo preguntas	54
14.9 Personalización desde el panel administrativo	54
14.10 Personalización de reglas y constantes	55
14.11 Aplicar cambios de configuración	55
15. Seguridad.....	56

15.1 Autenticación y sesiones.....	56
15.2 Roles y permisos de acceso	57
15.3 Seguridad del cliente tótem	58
15.4 Seguridad del backend y API.....	58
15.5 Seguridad de la base de datos	59
15.6 Seguridad de variables de entorno y secretos	60
15.7 Seguridad de archivos y contenidos	61
15.8 Seguridad de comunicación	61
15.9 Seguridad en Docker y despliegue.....	61
15.10 Respaldo y recuperación	62
16. Glosario de términos.....	62
17. Referencias y recursos adicionales	65
17.1 Documentación interna del proyecto	65
17.2 Frontend	66
17.3 Backend y API.....	66
17.4 Base de datos.....	67
17.5 Seguridad y autenticación	67
17.6 Archivos y PDFs	68
17.7 Servicios externos y APIs	68
17.8 Despliegue y hosting.....	68
17.9 Herramientas de desarrollo	69
17.10 Herramientas de apoyo	69
17.11 Foros y soporte comunitario	70
18. Herramientas de Implementación	70
18.1 Lenguajes de programación.....	70
18.2 Frameworks y tecnologías principales	70
18.3 Librerías principales del backend.....	71
18.4 Librerías principales del frontend.....	72
18.5 APIs y servicios externos	72
18.6 Herramientas de desarrollo	72
18.7 Herramientas de base de datos	73
18.8 Herramientas de despliegue	73

19. Bibliografía	73
------------------------	----

1. Integrantes – Roles

En el desarrollo del proyecto **TOTEM**, cada integrante del equipo cumplió un rol específico para organizar mejor el trabajo y distribuir las responsabilidades.

- **Alex Bryam Barrera Alarcon – Team Leader / QA / Developer**
 - Encargado de coordinar al equipo, organizar tareas y realizar pruebas para verificar el funcionamiento del sistema. También participo en el desarrollo del proyecto.
- **Rommel Rodrigo Gutierrez Herrera – DB Máster / Developer**
 - Responsable de la base de datos, incluyendo el diseño de tablas, relaciones, migraciones y consultas necesarias. Además, realizo el despliegue de la base de datos. También apoyo en el desarrollo del proyecto.
- **Mauricio Waly Linaja Canty – Developer**
 - Apoyo en el desarrollo del proyecto, colaborando en la implementación de funcionalidades, revisión del sistema y corrección de errores durante el proceso de desarrollo.
- **Edson Marcelo Cayo Ali – Git Máster / Developer**
 - Encargado de administrar el repositorio del proyecto controlar ramas, revisar cambios y apoyar en la integración del código. También participo en el desarrollo del sistema.

2. Introducción:

TOTEM es un sistema web orientado a la administración y visualización de contenido en tótems digitales. Su propósito es facilitar la gestión de información institucional, permitiendo mostrar imágenes, videos, noticias, y otros contenidos de manera organizada en diferentes dispositivos.

Además, el sistema incorpora un modo de preguntas, mediante el cual los usuarios pueden interactuar con el tótem para realizar consultas y recibir respuestas relacionadas con la información cargada en la plataforma.

El sistema busca mejorar la comunicación visual dentro de espacios institucionales, ofreciendo una plataforma centralizada para controlar y actualizar el contenido mostrado en los tótems de forma sencilla, eficiente e interactiva.

3. Descripción / objetivo del proyecto:

El proyecto **TOTEM** tiene como objetivo centralizar la gestión de contenido digital que se presenta en tótems informativos. A través de una plataforma web, los administradores pueden controlar que contenido se muestra, en que dispositivo y durante que periodo de tiempo.

El sistema permite la gestión de usuarios, tótems, contenidos, asignaciones y notificaciones, manteniendo una administración ordenada de la información publicada. También incluye un modo de preguntas, que permite a los usuarios consultar información directamente desde el tótem, mejorando la interacción y el acceso a los contenidos institucionales.

De esta manera, **TOTEM** contribuye a una comunicación institucional más dinámica, accesible, actualizada e interactiva.

4. Link al Video ilustrativo en YouTube

- <https://youtu.be/srlkGI-IKNA>

5. Listado de los Requisitos Funcionales del Sistema

A continuación, se presentan los requisitos funcionales del sistema **TOTEM**, organizados según módulos principales del panel administrativo y del cliente tótem.

RF-01. Autenticación y gestión de sesión

El sistema debe permitir el acceso al panel administrativo mediante un formulario de inicio de sesión, donde los usuarios con rol Admin o SuperAdmin ingresen su correo electrónico y contraseña.

Además, el sistema debe permitir cerrar sesión desde el panel administrativo para finalizar el acceso del usuario de forma segura.

RF-02. Control de acceso por roles y campus.

El sistema debe manejar dos roles principales:

Rol	Permisos principales
SuperAdmin	Puede visualizar y administrar la información de todos los campus, usuarios, tótems, contenidos, asignaciones y notificaciones.
Admin	Solo puede visualizar y administrar la información correspondiente a su campus asignado.

El control de acceso por campus debe aplicarse en todos los módulos del panel administrativo.

RF-03. Gestión de usuarios

El sistema debe permitir al usuario con rol SuperAdmin gestionar los usuarios del sistema.

El módulo de usuarios debe permitir:

- Crear usuarios.
- Editar usuarios.
- Eliminar usuarios.
- Ver detalles de usuarios.
- **Asignar rol:** Admin o SuperAdmin.
- Asignar campus.
- **Definir estado:** activo o inactivo.
- Registrar nombre y correo electrónico.

Al crear un nuevo usuario, el sistema debe enviar un correo de invitación para que el usuario pueda definir su contraseña de acceso.

RF-04. Invitación de usuarios

El sistema debe enviar un correo de invitación al usuario creado, incluyendo un enlace o botón que lo redirija a una página para establecer su contraseña.

El enlace de invitación debe tener una duración de 24 horas.

En caso de que el enlace expire o el usuario aun no haya activado su cuenta, el

SuperAdmin debe poder reenviar la invitación.

La contraseña definida por el usuario debe cumplir con los siguientes requisitos mínimos:

- Tener al menos 8 caracteres.
- Incluir una letra mayúscula.
- Incluir una letra minúscula.
- Incluir un número.
- Incluir un carácter especial

RF-05. Cambio y recuperación de contraseña

El sistema debe permitir que los usuarios cambien su contraseña desde el panel administrativo, ingresando:

- Contraseña actual.
- Nueva contraseña.
- Confirmación de la nueva contraseña.

También debe existir una opción de “¿Olvidaste tu contraseña?” en el inicio de sesión.

Al ingresar un correo registrado, el sistema debe enviar un enlace para restablecer la contraseña.

El enlace de recuperación debe expirar en 30 minutos.

Luego de establecer la nueva contraseña, el usuario debe ser redirigido al login para acceder nuevamente al sistema.

RF-06. Gestión de tótems

El sistema debe permitir gestionar los tótems digitales registrados en la plataforma.

El módulo de tótems debe permitir:

- Crear tótems.
- Editar tótems.
- Eliminar tótems.
- Ver detalles de tótems.
- Gestionar código de vinculación para el cliente tótem.

Cada tótem debe registrar la siguiente información:

- Código.
- Nombre.
- Campus.
- **Estado:** activo o inactivo.

El código de vinculación solo debe estar disponible para tótems activos y debe tener una duración limitada, recomendada de 5 o 10 minutos, antes de expirar.

En el caso del usuario **SuperAdmin**, el sistema debe permitir seleccionar el campus del tótem. Para el usuario **Admin**, el campus debe ignorarse automáticamente según su cuenta.

RF-07. Gestión de contenidos

El sistema debe permitir administrar los contenidos que serán mostrados en los tótems digitales.

El módulo de contenidos debe permitir:

- Crear contenidos.
- Editar contenidos.
- Eliminar contenidos.
- Ver detalles de contenidos.

Al crear un contenido, el sistema debe solicitar:

- Título.
- Tipo de contenido.
- Campus, solo en caso de usuario **SuperAdmin**.
- Descripción.
- Archivo del contenido, cuando corresponda.

Los tipos de contenido permitidos son:

- Imagen.
- Video.
- Publicidad.
- Noticia.
- PDF.

En el caso del contenido de tipo noticia, la descripción deber ser obligatoria y el archivo debe ser opcional.

RF-08. Gestión de preguntas y respuestas en contenidos PDF

El sistema debe permitir trabajar con preguntas y respuesta extraídas desde contenidos de tipo **PDF**.

Cuando un contenido sea de tipo PDF, el sistema debe extraer y mostrar las preguntas y respuestas asociadas en la vista de detalles del contenido.

Además, los contenidos PDF deben contar con una acción llamada **“Gestionar imágenes de preguntas”**, donde se visualicen las preguntas extraídas y se permitan asociar imágenes a cada una.

Cada pregunta puede tener imágenes asociadas de forma opcional, con un máximo de 4 imágenes por pregunta.

RF-09. Gestión de asignaciones

El sistema debe permitir asignar contenidos a uno o varios tótems, indicando cuando y como deben mostrarse.

El módulo de asignaciones debe permitir:

- Crear asignaciones.
- Editar asignaciones.
- Eliminar asignaciones.
- Ver detalles de asignaciones.

Al crear una asignación, el sistema debe permitir seleccionar:

Un tótem, varios tótems o todos los tótems disponibles.

- Uno o varios contenidos.
- Estado de la asignación.
- Prioridad.
- Orden de visualización.
- Rango de fecha de visualización.

El rango de fechas debe ser opcional.

El orden de visualización puede generarse automáticamente según el tótem y el tipo de contenido.

Cada tótem debe respetar un límite máximo de contenidos asignados por tipo:

Tipo de contenido	Limite por tótem
Imágenes	8
Videos	3
Publicidades	5
Noticias	6
PDFs	3

El sistema debe evitar asignaciones duplicadas o con conflictos de fechas para el mismo tótem y contenido.

RF-10. Gestión de notificaciones

El sistema debe permitir crear y administrar notificaciones que serán mostradas en los tótems.

El módulo de notificaciones debe permitir:

- Crear notificaciones.
- Editar notificaciones.
- Eliminar notificaciones.
- Ver detalles de notificaciones.

Al crear una notificación, el sistema debe solicitar:

- Título.
- Mensaje.
- Tipo de notificación.
- Estado.
- Duración.
- Fecha de inicio opcional.

Las notificaciones pueden ser de tipo:

- Normal.

- Urgente.

El sistema debe permitir enviar notificaciones a:

- Todos los tótems.
- Tótems de un campus específico, en caso de usuario **SuperAdmin**.
- Tótems seleccionados manualmente.

Para el usuario **Admin**, las notificaciones solo deben afectar a los tótems de su campus.

Cuando una notificación sea enviada, debe visualizarse en los tótems correspondientes.

RF-11. Dashboard administrativo

El sistema debe contar con un dashboard principal que resuma información importante del sistema.

El dashboard debe mostrar información relacionada con:

- Tótems registrados.
- Contenidos registrados.
- Asignaciones activas.
- Notificaciones.
- Usuarios, solo para el rol **SuperAdmin**.

También debe incluir alertas operativas, como:

- Tótems activos sin conexión.
- Asignaciones próximas a expirar.
- Invitaciones de usuarios pendientes.
- Otros avisos importantes para la administración del sistema.

RF-12. Visualización en el cliente tótem

Cada tótem debe contar con una interfaz cliente encargada de mostrar los contenidos asignados desde el panel administrativo.

El cliente tótem debe permitir vincularse con el servidor mediante un código de vinculación generado desde el panel administrativo.

Una vez vinculado, el tótem debe cargar únicamente los contenidos asignados a su dispositivo.

La visualización del tótem debe cumplir con lo siguiente:

- Mostrar contenido en pantalla completa.
- Organizar cada tipo de contenido en su sección correspondiente.
- Mostrar un mensaje de bienvenida si no existen contenidos asignados.
- Mostrar notificaciones normales como banner superior.
- Mostrar notificaciones urgentes en una pantalla de emergencia.
- Mostrar una sección de clima en la parte superior.

El cliente tótem debe contar con tres modos principales.

Modo	Descripción
Modo normal	Muestra los contenidos asignados al tótem.
Modo preguntas	Permite al usuario realizar consultas por voz.
Modo emergencia	Muestra notificaciones urgentes de forma destacada.

RF-13. Interacción mediante cámara y gestos

El sistema debe permitir que el cliente tótem utilice una cámara web para detectar gestos.

Al detectar un gesto definido, como una mano abierta, el tótem debe ingresar al modo preguntas y mostrar mensaje como:

- “Pregúntame lo que quieras”.
- “Escuchando...”.

Mientras el tótem se encuentre en modo de preguntas, la detección de gestos debe deshabilitarse temporalmente para evitar activaciones o errores de funcionamiento.

RF-14. Preguntas por voz

En el modo de preguntas, el usuario debe poder interactuar con el tótem mediante voz.

El sistema debe realizar las siguientes acciones:

- Capturar la voz a texto.
- Identificar la pregunta realizada.
- Buscar una respuesta en los documentos PDF asignados al tótem.
- Responder mediante texto a voz.
- Mostrar imágenes asociadas si la pregunta tiene imágenes registradas.

Si el tótem no tiene documentos PDF asignados, debe informar al usuario que no existen preguntas disponibles para responder en ese momento.

Si la pregunta no es válida o no se encuentra una respuesta relacionada, el sistema debe solicitar al usuario que realice una pregunta más específica.

Cuando la pregunta tenga imágenes asociadas, el tótem debe mostrarlas en un carrusel. Si no existen imágenes asociadas, solo debe responder mediante texto a voz y continuar escuchando nuevas preguntas.

RF-15. Retorno automático al modo normal

El sistema debe permitir que el cliente tótem regrese automáticamente al modo normal después de un tiempo de inactividad en el modo preguntas.

Este comportamiento evita que el tótem permanezca innecesariamente en modo preguntas y permite que continúe mostrando los contenidos asignados de manera normal.

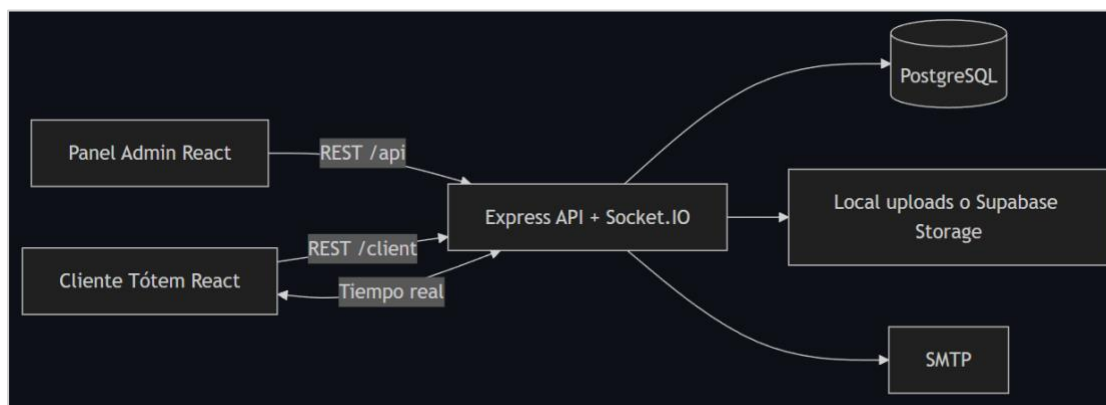
6. Arquitectura del software

El sistema **TOTEM** está diseñado bajo una arquitectura cliente-servidor, separando la interfaz de usuario, la lógica de negocio y el acceso a datos. Esta organización permite que el sistema sea más ordenado, mantenible y escalable.

El proyecto se encuentra estructurado principalmente en dos partes:

Componente	Descripción
Frontend	Aplicación desarrollada en React, TypeScript y Vite. Incluye el panel administrativo y la interfaz del cliente tótem.
Backend	API desarrollada con Node.js, Express y Sequelize. Gestiona la lógica del sistema, autenticación, permisos, archivos, notificaciones y comunicación con la base de datos.
Base de datos	PostgreSQL, utilizada para almacenar usuarios, campus, tótems, contenidos, asignaciones, notificaciones, sesiones y datos relaciones con preguntas PDF.
Almacenamiento de archivos	Puede trabajar con archivos locales o con Supabase Storage, según la configuración del sistema.
SMTP	Servicio utilizado para el envío de correos, como invitaciones de usuarios y recuperación de contraseña.
Socket.IO	Permite comunicación en tiempo real entre el backend y los clientes tótem.

6.1 Diagrama de arquitectura



6.2 Estructura general del sistema

Ruta – PR2-26-TOTEM/	Descripción
frontend/	Interfaz web administrativa y cliente tótem.

backend/	API REST, servicios, lógica de negocio y acceso a datos.
docker-compose.yml	Configuración de contenedores Docker.
README.md	Documentación del proyecto.

Esta estructura permite trabajar de forma independiente en cada capa del sistema, facilitando el mantenimiento, la integración y el despliegue.

6.3 Componente principales

Frontend

El frontend contiene dos interfaces principales:

- **Panel administrativo:** permite gestionar usuarios, campus, tótems, contenidos, asignaciones, notificaciones y visualizar el dashboard.
- **Cliente tótem:** se ejecuta en el dispositivo tótem, muestra los contenidos asignados, recibe notificaciones, permite la vinculación del dispositivo y habilita el modo de preguntas.

El frontend se comunica con el backend mediante peticiones HTTP a la API REST. En el caso del cliente tótem, también utiliza Socket.IO para recibir actualizaciones y notificaciones en tiempo real.

Backend

El backend centraliza la lógica principal del sistema. Sus responsabilidades más importantes son:

- Autenticación y gestión de sesiones.
- Control de acceso por roles y campus.
- Gestión de usuarios, tótems, contenidos, asignaciones y notificaciones.
- Administración de archivos,
- Procesamiento de documentos PDF para preguntas y respuesta.
- Comunicación en tiempo real con los tótems.

- Conexión con la base de datos PostgreSQL.
- Envío de correos mediante SMTP.

El backend se organiza en capas, separando rutas, controladores, servicios, repositorios, modelo, validaciones y middlewares.

Base de datos

La base de datos **PostgreSQL** almacena la información persistente del sistema. Entre los datos principales se encuentran:

- Usuarios.
- Campus.
- Tótems.
- Contenidos.
- Asignaciones.
- Notificaciones.
- Sesiones de dispositivos.
- Preguntas, respuesta e imágenes asociadas a PDFs.

Para la gestión de la base de datos se utiliza **Sequelize**, junto con modelos, migraciones y relaciones entre entidades.

6.4 Interacción entre componentes

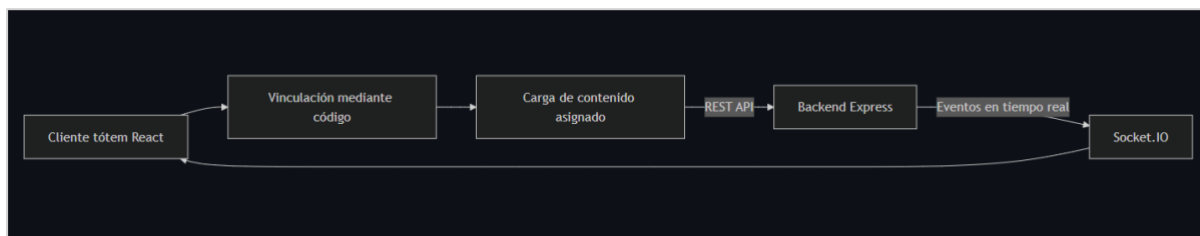
El flujo general del panel administrativo es el siguiente:



Por ejemplo, cuando un administrador crea una asignación de contenido, el frontend envía la solicitud al backend. Luego, el backend valida la sesión, verifica los permisos del usuario,

aplica las reglas de negocio, guarda la información en la base de datos y actualiza a los tótems correspondientes.

El flujo general del cliente tótem es el siguiente:



Esto permite que cada tótem cargue únicamente su contenido asignado, reporte su estado de conexión y reciba notificaciones o actualizaciones en tiempo real.

6.5 Patrones y organización utilizados

El sistema utiliza una arquitectura organizada por capas y varios patrones de apoyo:

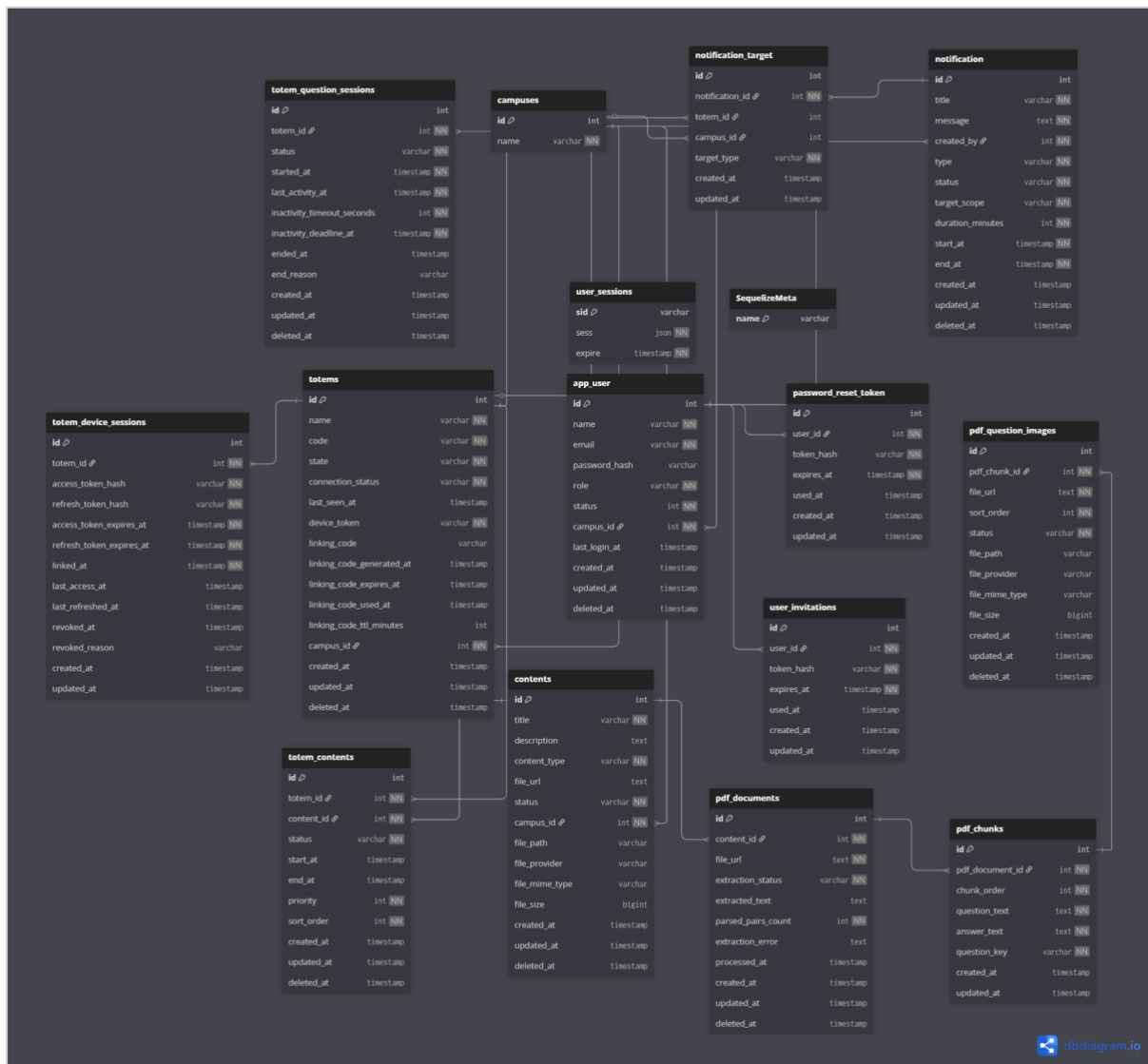
Patrón	Aplicación en el sistema
Arquitectura por capas	Separa rutas, controladores, servicios, repositorios y modelos en el backend.
MVC adaptado	Los controladores reciben solicitudes, los modelos representan los datos y los servicios contienen la lógica principal.
Repository Pattern	Encapsula las operaciones de base de datos para evitar que la lógica de negocio dependa directamente de Sequelize.
Service Layer	Centraliza reglas de negocio como asignaciones, notificaciones, autenticación, sesiones y procesamiento de PDFs.
Middlewares	Se utilizan para autenticación, autorización, validación, manejo de archivos, seguridad y control de errores.
SPA con rutas protegidas	El frontend funciona como una aplicación de una sola página con rutas públicas y privadas según el rol del usuario.
Comunicación en tiempo real	Socket.IO permite enviar eventos importantes desde el backend hacia los tótems conectados.
Abstracción de almacenamiento	El sistema puede trabajar con archivos locales o Supabase Storage sin afectar la lógica principal.

En conjunto, esta arquitectura permite que **TOTEM** mantenga una separación clara entre la interfaz, la lógica del sistema, la persistencia de datos, el almacenamiento de archivos y la comunicación en tiempo real con los dispositivos tótem.

7. Base de datos

El Sistema TOTEM utiliza una base de datos PostgreSQL para almacenar la información principal del sistema. La base de datos se encuentra relacionada con el backend mediante Sequelize ORM, permitiendo trabajar con modelos, migraciones y relaciones entre entidades de forma organizada.

7.1 Diagrama completo y actual



7.2 Organización de scripts en el repositorio Git

Estructura de archivos

Ruta - PR2-26-TOTEM/database/	Descripción
01_schema.sql	Script de generación de la base de datos. Contiene tablas, claves primarias, claves foráneas, índices y tipos necesarios.
02_seed.sql	Script de inserción de datos de ejemplo para pruebas o demostración del sistema.
README_BD.md	Documento descriptivo sobre la base de datos y el uso de los scripts.

7.3 Tablas principales de la base de datos

Tabla	Descripción
campuses	Almacena las sedes o campus disponibles del sistema.
app_user	Almacena los usuarios administrativos del sistema.
totems	Registra los tótems digitales, su estado, campus y datos de vinculación.
contents	Almacena los contenidos que pueden mostrarse en los tótems.
totem_contents	Relaciona tótems con contenidos asignados.
notification	Registra las notificaciones creadas desde el panel administrativo.
notification_target	Define los destinos de una notificación: tótems o campus
pdf_documents	Almacena información de los documentos PDF procesados.
pdf_chunks	Guarda preguntas y respuestas extraídas desde documentos PDF.
pdf_question_images	Permite asociar imágenes a preguntas extraídas desde PDFs.
totem_device_sessions	Registra las sesiones de vinculación de los dispositivos tótem.
totem_question_sessions	Registra las sesiones del modo de preguntas en el cliente tótem.
password_reset_token	Almacena tokens para recuperación de contraseña.
user_invitations	Almacena tokens de invitación para usuarios nuevos.
user_sessions	Guarda sesiones de usuarios autenticados.

SequelizeMeta	Tabla interna utilizada por Sequelize para controlar migraciones ejecutadas.
---------------	--

7.4 Script simple

El siguiente script representa una carga básica de datos de ejemplo para el sistema **TOTEM**. Este script debe ejecutarse después de haber creado la estructura de la base de datos con el archivo 01_schema.sql.

```
-- Script simple de ejemplo - Base de datos TOTEM

BEGIN;

-- 1. Insertar campus de ejemplo
INSERT INTO campuses (name)
VALUES
    ('Cochabamba'),
    ('La Paz'),
    ('Santa Cruz');

-- 2. Insertar usuario SuperAdmin de ejemplo
INSERT INTO app_user
(name, email, password_hash, role, status, campus_id)
VALUES
(
    'Administrador General',
    'superadmin@totem.test',
    'HASH_DE_CONTRASENA',
    'SuperAdmin',
    1,
    1
);

-- 3. Insertar tótem de ejemplo
INSERT INTO totems
(name, code, state, connection_status, device_token, campus_id)
VALUES
(
    'Tótem Principal Cochabamba',
    'TOTEM-CBBA-001',
    'active',
    'offline',
    'device-token-demo',
    1
);
```

```

);

-- 4. Insertar contenido de ejemplo
INSERT INTO contents
(title, description, content_type, file_url, status, campus_id)
VALUES
(
    'Bienvenida Institucional',
    'Imagen de bienvenida para estudiantes y visitantes.',
    'image',
    '/uploads/contents/bienvenida.png',
    'active',
    1
);

-- 5. Asignar contenido al tótem
INSERT INTO totem_contents
(totem_id, content_id, status, priority, sort_order)
VALUES
(
    1,
    1,
    'active',
    1,
    1
);

-- 6. Insertar notificación de ejemplo
INSERT INTO notification
(title, message, created_by, type, duration_minutes, target_scope,
start_at, end_at, status)
VALUES
(
    'Aviso institucional',
    'Bienvenidos al sistema informativo TOTEM.',
    1,
    'normal',
    60,
    'all',
    NOW(),
    NOW() + INTERVAL '60 minutes',
    'active'
);

COMMIT;

```

Este script es una versión simplificada utilizada para mostrar la inserción básica de datos dentro del manual técnico. El script completo de generación de la base de datos y el script completo de datos de ejemplo se encuentran en la carpeta database/ del repositorio de Git del proyecto.

8. Listado de Roles más sus credenciales de todos los Admin / Users del sistema

El sistema **TOTEM** cuenta con usuarios administrativos con roles definidos para controlar el acceso a los módulos del panel administrativo. Los roles principales son **SuperAdmin** y **Admin**.

Las credenciales se dividen en dos grupos:

- **Credenciales del entorno desplegado:** utilizadas para acceder al sistema hosteado.
- **Credenciales de prueba:** generadas mediante el archivo 02_seed.sql para pruebas locales o demostración.

8.1 Acceso al sistema desplegado

Recurso	URL
Panel Administrativo TOTEM	https://totem-univalle.vercel.app/
Cliente tótem	https://totem-univalle.vercel.app/client/totem
Backend API	https://totem-backend-univalle.onrender.com

8.2 Roles del sistema

Rol	Descripción
SuperAdmin	Usuario con acceso completo al sistema. Puede gestionar usuarios, tótems, contenidos, asignaciones, notificaciones y visualizar información general de todos los campus.
Admin	Usuario con acceso limitado a su campus asignado. Puede gestionar la información correspondiente únicamente a su sede.

8.3 Credenciales del entorno desplegado

Las siguientes credenciales corresponden a los usuarios habilitados para acceder al sistema **TOTEM** en el entorno hosteado.

Usuario	Correo	Contraseña	Rol
Juan Perez	superadmin@gmail.com	1234	SuperAdmin
Roberto Carlos	admin@gmail.com	1234	Admin

8.4 Credenciales de prueba del archivo 02_seed.sql

Las siguientes credenciales se generan al ejecutar el archivo 02_seed.sql en una base de datos local o de prueba.

Usuario	Correo	Contraseña	Rol
Administrador General	superadmin@totem.test	Admin123*	SuperAdmin
Admin Cochabamba	admin.cbba@totem.test	Admin123*	Admin
Admin La Paz	admin.lpz@totem.test	Admin123*	Admin

8.5 Acceso al cliente tótem

El cliente tótem no utiliza correo ni contraseña como el panel administrativo. Para vincular un tótem, se utiliza un **código de vinculación temporal** generado desde el módulo de tótems en el panel administrativo.

Elemento	Descripción
Método de acceso	Código de vinculación generado desde el panel administrativo.
Duración del código	Tiempo limitado, por ejemplo 5 o 10 minutos.
Uso	Permite asociar el dispositivo físico con un tótem registrado en el sistema.
Disponibilidad	Solo se puede generar para tótems activos.

Una vez vinculado, el cliente tótem mantiene una sesión propia para cargar sus contenidos asignados, reportar su estado de conexión y recibir notificaciones.

8.6 Enlace y recursos del proyecto

Recurso	Descripción	Enlace
---------	-------------	--------

GitHub	Repositorio del proyecto TOTEM.	https://github.com/CB-PROYECTO-SISTEMAS-2026/PR2-26-TOTEM
Figma	Diseño de interfaces del sistema.	https://www.figma.com/design/1fHSkxh2uEiX3geoezNigM/Untitled?node-id=0-1&t=EWQSSp2qbJQmjJYu-0
Linear	Gestión de tareas, issues y planificación del proyecto.	https://linear.app/pr-26-tottem/team/PR/all
Imagen Docker	Imagen relacionada con Docker del proyecto.	totem-docker-images.tar

9. Requisitos del sistema

Esta sección describe los requisitos mínimos necesarios para ejecutar el sistema **TOTEM**, considerando el uso desde el panel administrativo, el cliente tótem, el servidor backend, el hosting y la base de datos.

9.1 Requerimientos de Hardware – Cliente

El sistema cuenta con dos tipos de cliente: el usuario administrativo que accede al panel web y el dispositivo tótem que muestra el contenido público.

Cliente	Requisito mínimo
Panel administrativo	Computadora de escritorio o portátil con procesador de 2 núcleos, 4 GB de RAM y pantalla con resolución mínima de 1366 x 768.
Cliente tótem	Mini PC, computadora o dispositivo equivalente con procesador de 4 núcleos, 4 GB de RAM, salida de video y pantalla conectada.
Pantalla del tótem	Pantalla o monitor con resolución mínima HD. Se recomienda Full HD para mejor visualización de imágenes, videos, noticias y mensajes.
Almacenamiento local del cliente	Espacio suficiente para el sistema operativo y navegador. El cliente no almacena la base de datos ni los archivos principales del sistema.
Red	Conexión estable a internet o a la red donde se encuentra publicado el backend.

Cámara	Necesaria en el cliente tótem para detectar gestos y activar el modo preguntas.
Micrófono y parlantes	Necesarios para utilizar el modo preguntas por voz y la respuesta mediante audio.

9.2 Requerimientos de Software – Cliente

Software	Requisito
Sistema operativo	Windows, Linux o macOS para el panel administrativo. Para el tótem puede usarse cualquier sistema operativo capaz de ejecutar un navegador moderno.
Navegador web	Google Chrome o Microsoft Edge actualizado. También puede funcionar en otros navegadores modernos compatibles con JavaScript, WebSocket y APIs multimedia.
JavaScript	Debe estar habilitado en el navegador.
Cookies	Deben estar habilitadas para mantener la sesión del panel administrativo.
WebSocket	Debe estar permitido para recibir eventos en tiempo real mediante Socket.IO.
Permisos multimedia	El navegador debe permitir el acceso a cámara y micrófono cuando se utilice el modo preguntas.
HTTPS	En producción se recomienda acceder mediante HTTPS para permitir el uso seguro de cámara, micrófono, sesiones y comunicación con el backend.

9.3 Requerimientos de Hardware – Servidor / Hosting / Base de Datos

Los siguientes requisitos aplican al servidor donde se ejecuta el backend, el frontend y la base de datos. En caso de utilizar servicios externos, como hosting para frontend, backend, almacenamiento o base de datos, los recursos pueden distribuirse entre cada servicio.

Recurso	Requisito mínimo
Procesador	2 vCPU o procesador de 2 núcleos.
Memoria RAM	4 GB de RAM como mínimo.

Almacenamiento	20 GB disponibles para aplicación, base de datos, logs y archivos subidos.
Almacenamiento persistente	Requerido para PostgreSQL y para archivos si se utiliza almacenamiento local.
Red	Conexión estable y puertos habilitados para frontend, backend y base de datos.
Escalabilidad de disco	El espacio debe poder ampliarse según la cantidad de imágenes, videos, PDFs, registros y archivos almacenados.

En caso de usar una base de datos externa, como Render PostgreSQL, el servidor del backend no necesita alojar PostgreSQL localmente, pero si debe tener acceso de red y configuración correcta para conectarse con la base de datos.

9.4 Requerimientos de Software – Servidor / Hosting / Base de Datos

Software	Requisito
Sistema operativo	Linux, Windows Server o entorno compatible con Node.js y PostgreSQL. Para despliegue con contenedores se recomienda Linux o Docker Desktop.
Node.js	Versión 22 o compatible con el backend del proyecto.
npm	Necesario para instalar dependencias y ejecutar scripts del frontend y backend cuando no se utiliza Docker.
PostgreSQL	Versión 16 o compatible.
Sequelize CLI	Utilízalo para ejecutar migraciones y seeders de la base de datos.
Docker	Opcional, requerido si se ejecuta el sistema mediante contenedores.
Docker Compose	Opcional, requerido para levantar frontend, backend y PostgreSQL mediante docker-compose.yml
Nginx	Utilizado en el contenedor del frontend para servir la aplicación compilada y redirigir peticiones hacia el backend.
SMTP	Requerido para el envío de correos de invitación de usuarios y recuperación de contraseña.
Almacenamiento de archivos	Puede ser almacenamiento local del servidor o Supabase Storage, según la configuración del sistema.

Certificado SSL/TLS	Recomendado en producción para proteger credenciales, sesiones, cámara, micrófono y comunicación con el backend.
---------------------	--

9.5 Requerimientos del Proyecto

El sistema **TOTEM** requiere las siguientes tecnologías para su funcionamiento:

Componente	Tecnología utilizada
Frontend	React, TypeScript, Vite y TailwindCSS.
Backend	Node.js, Express.js, Sequelize y Socket.IO.
Base de datos	PostgreSQL.
ORM	Sequelize.
Comunicación en tiempo real	Socket.IO.
Gestión de archivos	Almacenamiento local o Supabase Storage.
Correo electrónico	Servicio SMTP.
Modo preguntas	Cámara, micrófono, reconocimiento de voz, texto a voz y procesamiento de documentos PDF.

9.6 Variables de Entorno Necesarias

Para ejecutar el sistema correctamente, se deben configurar variables de entorno en el backend y frontend.

Entre las principales variables requeridas se encuentran:

Categoría	Variables relacionadas
Servidor	Puerto del backend, URL del frontend y configuración de CORS.
Base de datos	Host, puerto, nombre de base de datos, usuario, contraseña y configuración SSL si corresponde.
Sesiones	Secreto de sesión, nombre de cookie y tiempo de expiración.
Correo SMTP	Host, puerto, usuario, contraseña y configuración de seguridad.
Archivos	Proveedor de almacenamiento, ruta local o configuración de Supabase Storage.
Frontend	URL base de la API o rutas relativas según el entorno de despliegue.

9.7 Requisitos para Ejecución de Docker

Si el sistema se ejecuta mediante Docker Compose, se requiere.

Requisito	Descripción
Docker instalado	Docker Desktop o Docker Engine.
Docker Compose	Necesario para levantar los servicios definidos en docker-compose.yml.
Archivo .env	Contiene la configuración necesaria para los servicios.
Puertos disponibles	Puertos para frontend, backend y PostgreSQL, según la configuración.
Volúmenes persistentes	Para conservar datos de PostgreSQL y archivos subidos.

Con Docker Compose, el sistema puede levantar los servicios principales: PostgreSQL, backend Express y frontend React desde un solo comando.

10. Instalación y configuración

Esta sección describe los pasos principales para instalar, configurar y ejecutar el sistema **TOTEM**, tanto en entorno local como en un entorno desplegado. El sistema está compuesto por un frontend React, un backend Node.js/Express, una base de datos PostgreSQL y servicios externos opcionales como SMTP y almacenamiento de archivos.

10.1 Requisitos previos

Antes de instalar el sistema, se debe contar con las siguientes herramientas:

Componente	Requisito
Node.js	Versión 22 o compatible.
npm	Instalado junto con Node.js.
PostgreSQL	Versión 16 o compatible, en local o en un servicio externo como Render PostgreSQL.
Git	Para clonar o descargar el repositorio.
Docker y Docker Compose	Opcional, si se desea ejecutar el sistema mediante contenedores.
SMTP	Requerido para invitaciones de usuarios y recuperación de contraseña.
Almacenamiento	Local o Supabase Storage, según la configuración del sistema.

10.2 Estructura general del proyecto

El proyecto se organiza en dos componentes principales: backend y frontend.

Ruta – PR2-26-TOTEM/	Descripción
frontend/	Panel administrativo y cliente público del tótem.
backend/	API REST, autenticación, Socket.IO, conexión a BD y archivos.
database/	Scripts SQL de base de datos
docker-compose.yml	Configuración de contenedores Docker.
README.md	Documentación del proyecto.

10.3 Instalación local

1. Obtener el proyecto

```
git clone https://github.com/CB-PROYECTO-SISTEMAS-2026/PR2-26-TOTEM.git
cd PR2-26-TOTEM
```

Si el proyecto fue entregado como archivo comprimido, se debe descomprimir y abrir la carpeta raíz.

2. Configurar la base de datos.

Crear una base de datos PostgreSQL para el sistema:

```
CREATE DATABASE totem;
CREATE USER totem_user WITH PASSWORD 'totem_password';
GRANT ALL PRIVILEGES ON DATABASE totem TO totem_user;
ALTER DATABASE totem OWNER TO totem_user;
```

También se puede usar una base de datos remota, por ejemplo, Render PostgreSQL, configurando correctamente las variables de conexión en el backend.

3. Configurar el backend

Entrar a la carpeta del backend:

```
cd backend  
npm install
```

Crea el archivo de variables de entorno:

```
copy .env.example .env
```

En Linux o macOS:

```
cp .env.example .env
```

Ejemplo básico de configuración para backend/.env:

```
PORT=3000  
NODE_ENV=development  
FRONTEND_URL=http://localhost:5173  
  
PGHOST=localhost  
PGPORT=5432  
PGDATABASE=totem  
PGUSER=totem_user  
PGPASSWORD=totem_password  
PGSSL=false  
PGSSLMODE=disable  
  
SESSION_SECRET=clave_segura_para_sesiones  
SESSION_COOKIE_NAME=totem.sid  
SESSION_MAX_AGE_MS=28800000  
  
STORAGE_PROVIDER=local  
LOCAL_UPLOAD_DIR=uploads  
PUBLIC_FILES_URL=http://localhost:3000/uploads  
  
SMTP_HOST=smtp.gmail.com  
SMTP_PORT=465  
SMTP_SECURE=true  
SMTP_USER=correo@example.com  
SMTP_PASS=password_de_aplicacion  
SMTP_FROM=TOTEM <correo@example.com>
```

4. Ejecutar migraciones y datos iniciales

Desde la carpeta backend/, ejecutar:

```
npm run db:migrate
```

Para crear o actualizar el usuario inicial **SuperAdmin**:

```
npm run db:seed:admin
```

Si se desea cargar datos de ejemplo desde los scripts SQL, se pueden ejecutar los archivos ubicados en la carpeta database/:

```
psql -U totem_user -d totem -f database/01_schema.sql  
psql -U totem_user -d totem -f database/02_seed.sql
```

5. Iniciar el backend

```
npm run dev
```

El backend quedara disponible en:

- <http://localhost:3000>

Rutas de verificación:

- <http://localhost:3000/api/health>
- <http://localhost:3000/api/db-test>

6. Configurar el frontend

Abrir otra terminal desde la raíz del proyecto:

```
cd frontend  
npm install
```

Crea el archivo de variables de entorno:

```
copy .env.example .env
```

En Linux o macOS:

```
cp .env.example .env
```

Ejemplo básico para frontend/.env:

```
VITE_APP_NAME=TOTEM  
VITE_API_URL=http://localhost:3000
```

```
VITE_AUTH_ME_TIMEOUT_MS=12000
```

7. Iniciar el frontend

Desde la carpeta frontend/, ejecutar:

```
npm run dev
```

El frontend quedara disponible en:

- <http://localhost:5173>

10.4 Instalación con Docker Compose

El sistema también puede ejecutarse mediante Docker Compose, levantando los servicios principales desde un solo comando.

1. Configurar variables

Desde la raíz del proyecto:

```
copy .env.example .env
```

En Linux o macOS:

```
cp .env.example .env
```

Variables principales:

```
POSTGRES_PORT=5432
BACKEND_PORT=3000
FRONTEND_PORT=5173

POSTGRES_DB=totem
POSTGRES_USER=totem
POSTGRES_PASSWORD=totem_dev_password_change_me

FRONTEND_URL=http://localhost:5173
PUBLIC_FILES_URL=/uploads
SESSION_SECRET=dev_session_secret_change_me
```

2. Levantar los servicios

```
docker compose up --build
```

Servicios disponibles:

Servicio	URL o puerto
Frontend	http://localhost:5173
Backend	http://localhost:3000
PostgreSQL	localhost:5432
Health backend	http://localhost:3000/api/health
Prueba de BD	http://localhost:3000/api/db-test

3. Ejecutar migraciones en Docker

```
docker compose exec backend npm run db:migrate
```

Crear o actualizar usuario inicial:

```
docker compose exec backend npm run db:seed:admin
```

4. Detener servicios

```
docker compose down
```

Para eliminar también volúmenes persistentes:

```
docker compose down -v
```

- **Advertencia:** docker compose down -v elimina los datos de PostgreSQL y los archivos subidos en los volúmenes de Docker.

10.5 Configuración de servicios externos

Base de datos remota

En el entorno desplegado, la base de datos del sistema **TOTEM** se encuentra alojada en **Render PostgreSQL**. El backend desplegado en Render se conecta a esta base de datos mediante variables de entorno configuradas en el panel del servicio.

Variables de conexión del backend:

```
PGHOST=<HOST_DE_RENDER>
PGPORT=<PUERTO>
PGDATABASE=<NOMBRE_BD>
PGUSER=<USUARIO>
PGPASSWORD=<CONTRASEÑA>
PGSSL=true
PGSSLMODE=require
```

Correo SMTP

El servicio SMTP se utiliza para enviar correos de invitación y recuperación de contraseña.

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=465
SMTP_SECURE=true
SMTP_USER=correo@example.com
SMTP_PASS=password_de_aplicacion
SMTP_FROM=TOTEM <correo@example.com>
```

Almacenamiento de archivos

Para almacenamiento local:

```
STORAGE_PROVIDER=local
LOCAL_UPLOAD_DIR=uploads
PUBLIC_FILES_URL=http://localhost:3000/uploads
```

Para Supabase Storage:

```
STORAGE_PROVIDER=supabase
SUPABASE_URL=https://proyecto.supabase.co
SUPABASE_SERVICE_ROLE_KEY=clave_service_role
SUPABASE_STORAGE_BUCKET=totem-contents
PUBLIC_FILES_URL=https://proyecto.supabase.co/storage/v1/object/public/totem-contents
```

10.6 Configuración del sistema desplegado

En el entorno hosteado, el sistema puede estar distribuido de la siguiente forma:

Componente	Servicio sugerido
Frontend	Vercel.
Backend	Render.
Base de datos	Render PostgreSQL.

Archivos	Supabase Storage o almacenamiento local del servidor.
Correo	SMTP.

URLs del entorno desplegado:

Recurso	URL
Panel administrativo	https://totem-univalle.vercel.app/
Cliente totem	https://totem-univalle.vercel.app/client/totem
Backend API	https://totem-backend-univalle.onrender.com
Health backend	https://totem-backend-univalle.onrender.com/api/health
Prueba de base de datos	https://totem-backend-univalle.onrender.com/api/db-test

10.7 Credenciales de acceso

Acceso a la aplicación TOTEM

Entorno	Rol	URL	Correo	Contraseña
Producción	SuperAdmin	https://totem-univalle.vercel.app/	superadmin@gmail.com	1234
Producción	Admin	https://totem-univalle.vercel.app/	admin@gmail.com	1234
Pruebas locales	SuperAdmin	http://localhost:5173	superadmin@totem.test	Admin123*
Pruebas locales	Admin	http://localhost:5173	admin.cbba@totem.test	Admin123*

Acceso a la base de datos

Entorno	Host	Puerto	Base de datos	Usuario	Contraseña
Local	localhost	5432	totem	totem_user	totem_password
Docker	localhost	5432	totem	totem	totem_dev_password_change_me

Producción:

- **Host:** dpg-d84ec48g4nts73f53hfg-a.oregon-postgres.render.com
- **Puerto:** 5432.

- **Base de datos:** totem_db.
- **Usuario:** totem_user.
- **Contraseña:** Entregada por canal privado.

Acceso al host

Si el sistema esta desplegado en servicios como Vercel o Render, normalmente no se utiliza acceso por SSH, sino acceso mediante el panel web de cada plataforma.

Servicio	Método de acceso
Vercel	Panel web de Vercel con cuenta autorizada.
Render	Panel web de Render con cuenta autorizada.
Servidor propio	Acceso SSH, si corresponde.

En caso de ser servidor propio:

```
ssh usuario@IP_DEL_SERVIDOR
```

11. Procedimiento de hosteado / Hosting

Esta sección describe la puesta en marcha del sistema **TOTEM** en hosting, considerando el sitio web, backend, base de datos, almacenamiento de archivos, correo y servicios externos.

11.1 Distribución del sistema en hosting

Componente	Servicio utilizado	Función
Sitio web / Frontend	Vercel	Aloja el panel administrativo y el cliente tótem.
API / Backend	Render	Ejecuta la API Node.js/Express, Socket.IO y lógica del sistema.
Base de datos	Render PostgreSQL	Almacena usuarios, totems, contenidos, asignaciones, notificaciones y sesiones.
Archivos	Supabase Storage	Almacena imágenes, videos PDFs e imágenes asociadas a preguntas.
Correos	SMTP Brevo	Envía invitaciones de usuarios y recuperación de contraseña.

Clima	Open-Meteo API	Proporciona datos del clima para el cliente tótem.
-------	----------------	--

11.2 Sitio web en Vercel

El frontend se despliega en Vercel desde el repositorio del proyecto.

Pasos principales

- Ingresar al panel de Vercel.
- Importar el repositorio del proyecto.
- Seleccionar la carpeta del frontend.
- Configurar el comando de instalación y compilación.
- Agregar las variables de entorno.
- Ejecutar el despliegue.
- Verificar que el sitio cargue correctamente.

Configuración principal en Vercel

Configuración	Valor
Framework Preset	Vite
Root Directory	frontend
Install Command	npm install
Build Command	npm run build
Output Directory	dist
Environment	Production
Branch	main

Variables de entorno del frontend

```
VITE_APP_NAME=TOTEM
VITE_API_URL=https://totem-backend-univalle.onrender.com
VITE_AUTH_ME_TIMEOUT_MS=12000
```

URLs del frontend

Recurso	URL
Panel administrativo	https://totem-univalle.vercel.app/
Cliente tótem	https://totem-univalle.vercel.app/client/totem

11.3 API / Backend en Render

El backend se despliega en Render como servicio web. Este componente se encarga de la autenticación, conexión a base de datos, gestión de archivos, notificaciones, Socket.IO, correos y modo preguntas.

Pasos principales

- Ingresar al panel de Render.
- Crear un nuevo servicio web.
- Conectar el repositorio del proyecto.
- Seleccionar la carpeta del backend.
- Configurar las variables de entorno.
- Configurar el comando de instalación y ejecución.
- Ejecutar el despliegue.
- Verificar el estado del backend desde `/api/health`.

Configuración principal en Render

Configuración	Valor
Root Directory	backend
Build Command	<code>npm install --include=dev && npm run db:migrate</code>
Start Command	<code>npm start</code>
Environment	Node
Branch	main

Variables principales del backend en producción

```

PORT=3000
NODE_ENV=production
FRONTEND_URL=https://totem-univalle.vercel.app

PGHOST=<HOST_RENDER_POSTGRESQL>
PGPORT=5432
PGDATABASE=<NOMBRE_BD>
PGUSER=<USUARIO_BD>
PGPASSWORD=<CONTRASEÑA_BD>
PGSSL=true
PGSSLMODE=require
PGSSL_REJECT_UNAUTHORIZED=true

STORAGE_PROVIDER=supabase
SUPABASE_URL=<URL_SUPABASE>
SUPABASE_SERVICE_ROLE_KEY=<SERVICE_ROLE_KEY>
SUPABASE_STORAGE_BUCKET=totem-contents
PUBLIC_FILES_URL=<URL_PUBLICA_BUCKET>

SESSION_SECRET=<CLAVE_SEGURA>
SESSION_COOKIE_NAME=totem.sid
SESSION_MAX_AGE_MS=28800000
SESSION_TABLE_NAME=user_sessions
PASSWORD_SALT_ROUNDS=12

SMTP_HOST=<HOST_SMTP_BREVO>
SMTP_PORT=2525
SMTP_SECURE=false
SMTP_USER=<USUARIO_SMTP_BREVO>
SMTP_PASS=<CONTRASEÑA_SMTP_BREVO>
SMTP_FROM=TOTEM <correo@dominio.com>
EMAIL_APP_NAME=TOTEM
EMAIL_LOGO_URL=<URL_PUBLICA_LOGO>

```

URLs del backend

Recurso	URL
Backend API	https://totem-backend-univalle.onrender.com
Health backend	https://totem-backend-univalle.onrender.com/api/health
Prueba de base de datos	https://totem-backend-univalle.onrender.com/api/db-test

11.4 Base de datos en Render PostgreSQL

La base de datos se encuentra alojada en Render PostgreSQL. El backend se conecta con ella mediante variables de entorno configuradas en Render.

Pasos principales

- Crear una base de datos PostgreSQL en Render.
- Copiar los datos de conexión proporcionados por Render.

- Configurar esos datos en las variables de entorno del backend.
- Ejecutar migraciones o scripts SQL.
- Verificar la conexión a la base de datos desde /api/db-test.

Variables de conexión

```
PGHOST=<HOST_RENDER_POSTGRESQL>
PGPORT=5432
PGDATABASE=<NOMBRE_BD>
PGUSER=<USUARIO_BD>
PGPASSWORD=<CONTRASEÑA_BD>
PGSSL=true
PGSSLMODE=require
```

Scripts de base de datos

Los scripts de base de datos se encuentran en la carpeta:

```
database/
|-- 01_schema.sql
|-- 02_seed.sql
`-- README_BD.md
```

Script	Descripción
01_schema.sql	Crear la estructura de la base de datos.
02_seed.sql	Inserta datos de prueba o demostración.

También se pueden utilizar migraciones desde el backend:

```
npm run db:migrate
npm run db:seed:admin
```

11.5 Almacenamiento de archivos en Supabase Storage

Los archivos del sistema se almacenan en Supabase Storage. Esto incluye imágenes, videos, PDFs y archivos asociados a preguntas del modo interactivo.

Pasos principales

- Crear un proyecto en Supabase.
- Crear el bucket totem-contents.

- Configurar el bucket como público para la demostración.
- Copiar la URL del proyecto y la clave secreta.
- Configurar las variables en Render.
- Verificar la carga y visualización de archivos desde el sistema.

Variables de entorno

```
STORAGE_PROVIDER=supabase
SUPABASE_URL=<URL_SUPABASE>
SUPABASE_SERVICE_ROLE_KEY=<SERVICE_ROLE_KEY>
SUPABASE_STORAGE_BUCKET=totem-contents
PUBLIC_FILES_URL=<URL_PUBLICA_BUCKET>
```

11.6 Correos con SMTP Brevo

El sistema utiliza Brevo SMTP para enviar correos de invitación y recuperación de contraseña.

Pasos principales

- Crear o ingresar a una cuenta de Brevo.
- Obtener las credenciales SMTP.
- Configurar las variables SMTP en Render.
- Probar el envío de invitaciones o recuperación de contraseña.

Variables de entorno

```
SMTP_HOST=<HOST_SMTP_BREVO>
SMTP_PORT=2525
SMTP_SECURE=false
SMTP_USER=<USUARIO_SMTP_BREVO>
SMTP_PASS=<CONTRASEÑA_SMTP_BREVO>
SMTP_FROM=TOTEM <correo@dominio.com>
EMAIL_APP_NAME=TOTEM
EMAIL_LOGO_URL=<URL_PUBLICA_LOGO>
```

11.7 API de clima Open-Meteo

El cliente tótem utiliza Open-Meteo API para mostrar información del clima. Este servicio no requiere credenciales dentro del sistema.

Variables relacionadas:

```
TOTEM_WEATHER_CACHE_TTL_SECONDS=600
TOTEM_WEATHER_REQUEST_TIMEOUT_MS=4500
```

11.8 Credenciales de acceso

Acceso al sistema TOTEM

Rol	URL	Correo	Contraseña
SuperAdmin	https://totem-univalle.vercel.app/	superadmin@gmail.com	1234
Admin	https://totem-univalle.vercel.app/	admin@gmail.com	1234

Acceso a la base de datos

- **Host:** dpg-d84ec48g4nts73f53hfg-a.oregon-postgres.render.com
- **Puerto:** 5432.
- **Base de datos:** totem_db.
- **Usuario:** totem_user.
- **Contraseña:** Entregada por canal privado

Acceso al host

Como el sistema esta desplegado en servicios administrados, normalmente no se utiliza acceso SSH.

Servicio	Método de acceso
Vercel	Panel web de Vercel con cuenta autorizada.
Render	Panel web de Render con cuenta autorizada.
Supabase	Panel web de Supabase con cuenta autorizada.
Brevo	Panel web de Brevo con cuenta autorizada.

12. GIT

El proyecto **TOTEM** fue gestionado mediante Git como sistema de control de versiones. Durante el desarrollo se trabajó con una rama principal de integración llamada develop, donde se unían los avances realizados en las diferentes ramas de funcionalidades. Una vez

finalizado y validado el proyecto, la versión estable fue integrada en la rama main, quedando como la versión final entregada.

12.1 Rama final de entrega

Rama	Descripción
main	Contiene la versión final, estable e integrado del proyecto TOTEM . Esta es la rama utilizada para la entrega final.
develop	Rama de integración utilizada durante el desarrollo. En esta rama se unieron y probaron las funcionalidades antes de pasar a main.

La versión final del sistema fue integrada desde la rama develop hacia la rama main.

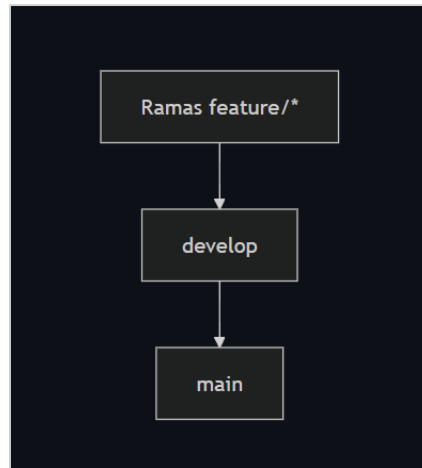
12.2 Ramas utilizadas durante el desarrollo

Durante el desarrollo del proyecto se utilizaron ramas separadas por funcionalidad, lo que permitió trabajar de forma más ordenada y reducir conflictos entre integrantes.

Rama	Propósito
main	Rama final de entrega del proyecto.
develop	Rama de integración general del desarrollo.
feature/admin-dashboard	Desarrollo del dashboard administrativo.
feature/admin-login	Desarrollo del inicio de sesión del panel administrativo.
feature/admin-password-recovery	Implementación de recuperación de contraseña.
feature/admin-role-management	Gestión de roles y permisos de usuarios.
feature/question-images-for-pdfs	Asociación de imágenes a preguntas extraídas de PDFs.
feature/supabase-local-storage	Implementación de almacenamiento local y Supabase Storage.
feature/totem-device-token	Vinculación y sesiones del dispositivo tótem.
feature/totem-management	Gestión de todo lo relacionado a los totems desde el panel administrativo.

12.3 Flujo de trabajo utilizado

El flujo de trabajo aplicado fue el siguiente:



Cada funcionalidad se trabajó en una rama independiente. Luego, los cambios se integraban en develop para realizar pruebas y validaciones. Finalmente, cuando el sistema quedó completo y estable, se integró develop en main para la entrega final.

12.4 Entrega del proyecto

La entrega final del proyecto se realizó desde la rama main, la cual contiene:

- Backend del sistema.
- Frontend del panel administrativo y cliente tótem.
- Configuración de hosting.
- Scripts de base de datos.
- Documentación técnica.
- Configuración de almacenamiento, SMTP y servicios externos.

En caso del frontend, los archivos compilados se generan mediante el comando:

```
npm run build
```

En el caso del backend, el sistema se ejecuta mediante:

```
npm start
```

12.5 Repositorio del proyecto

Elemento	Detalle
Repositorio	https://github.com/CB-PROYECTO-SISTEMAS-2026/PR2-26-TOTEM
Rama final	main
Rama de integración	develop
Control de versiones	Git
Plataforma	GitHub

La rama main representa la versión final entregada del sistema **TOTEM**.

13. Dockerizado Del Backend, Frontend y Base de Datos

El sistema **TOTEM** cuenta con una configuración mediante Docker Compose, permitiendo levantar en contenedores el backend, frontend y base de datos PostgreSQL desde un solo comando.

13.1 Componentes dockerizados

Servicio	Imagen/base	Puerto local	Función
postgres	postgres:16-alpine	5432	Base de datos PostgreSQL.
backend	node:22-bookworm-slim	3000	API REST, autenticación, Socket.IO, Sequelize y gestión de archivos.
frontend	nginx:1.27-alpine	5173	Aplicación React/Vite compilada y servida con Nginx.

La comunicación entre servicios se realiza dentro de la red interna de Docker Compose. El backend se conecta a PostgreSQL usando el nombre del servicio postgres.

13.2 Proceso de dockerizado

Backend

El backend se dockeriza desde la carpeta backend/.

Proceso general:

1. Se utiliza Node.js como imagen base.

2. Se copian los archivos package.json y package-lock.json,
3. Se instalan las dependencias.
4. Se copia el código fuente del backend.
5. Se expone el puerto 3000.
6. El servidor se inicia con npm start.

El backend utiliza las variables de entorno para conectarse a PostgreSQL:

```
PGHOST=postgres  
PGPORT=5432  
PGDATABASE=totem  
PGUSER=totem  
PGPASSWORD=totem_dev_password_change_me
```

Frontend

El frontend se dockeriza desde la carpeta frontend/.

Proceso general:

1. Se utiliza Node.js para compilar el proyecto React/Vite.
2. Se instalan las dependencias.
3. Se ejecuta npm run build.
4. Se genera la carpeta dist.
5. Se usa Nginx para servir los archivos compilados.
6. Nginx redirige las rutas necesarias hacia el backend.

En Docker, el frontend puede trabajar con rutas relativas, por eso la variable puede quedar vacía:

```
VITE_API_URL=
```

Base de datos

La base de datos se ejecuta mediante PostgreSQL en contenedor.

Configuración principal:

Campo	Valor
Servicio	postgres
Base de datos	totem
Usuario	totem
Puerto	5432
Volumen persistente	postgres_data

El volumen permite conservar los datos, aunque los contenedores sean detenidos.

13.3 Configuración principal

El archivo principal para levantar los servicios es:

- docker-compose.yml

Además, se utiliza un archivo .env en la raíz del proyecto para definir las variables de entorno.

Ejemplo de variables principales:

```
POSTGRES_PORT=5432
BACKEND_PORT=3000
FRONTEND_PORT=5173

POSTGRES_DB=totem
POSTGRES_USER=totem
POSTGRES_PASSWORD=totem_dev_password_change_me

NODE_ENV=development
FRONTEND_URL=http://localhost:5173

STORAGE_PROVIDER=local
LOCAL_UPLOAD_DIR=uploads
PUBLIC_FILES_URL=/uploads

SESSION_SECRET=dev_session_secret_change_me

VITE_APP_NAME=TOTEM
VITE_API_URL=
```

13.4 Como hacer correr el sistema

Desde la raíz del proyecto, ejecutar:

```
docker compose up --build
```

Para levantar los contenedores usando imágenes exportadas si se cuenta con el archivo totem-docker-images.tar:

```
docker load -i totem-docker-images.tar
docker compose up
```

Para levantar los contenedores en segundo plano:

```
docker compose up -d --build
```

Para verificar los contenedores activos:

```
docker compose ps
```

Para ver los logs:

```
docker compose logs -f
```

Para ver logs de un servicio específico:

```
docker compose logs -f backend
docker compose logs -f frontend
docker compose logs -f postgres
```

Para detener los contenedores:

```
docker compose down
```

Para detener y eliminar también los volúmenes:

```
docker compose down -v
```

- **Advertencia:** el comando `docker compose down -v` elimina los datos persistentes de PostgreSQL y los archivos subidos en los volúmenes de Docker.

13.5 Inicialización de la base de datos

Una vez levantados los contenedores, se debe ejecutar las migraciones desde el backend:

```
docker compose exec backend npm run db:migrate
```

Para crear o actualizar el usuario inicial SuperAdmin:

```
docker compose exec backend npm run db:seed:admin
```

También se pueden cargar los scripts SQL ubicados en la carpeta database/.

```
database/
|-- 01_schema.sql
|-- 02_seed.sql
`-- README_BD.md
```

Archivo	Descripción
01_schema.sql	Crea la estructura de la base de datos.
02_schema.sql	Inserta datos válidos y legibles para pruebas o demostración.

Para cargar los scripts SQL en Docker ejecutar en PowerShell:

Cambia:

- **-U totem** por el nombre de usuario que definiste en las variables de entorno.
- **-d totem** por el nombre de la base de datos que definiste en las variables de entorno.

```
Get-Content database/01_schema.sql | docker compose exec -T postgres
psql -U totem -d totem
Get-Content database/02_seed.sql | docker compose exec -T postgres psql
-U totem -d totem
```

Si la estructura ya fue creada con migraciones y solo se desean cargar datos de ejemplo:

```
Get-Content database/02_seed.sql | docker compose exec -T postgres psql -U totem
-d totem
```

13.6 Acceso a los servicios

Servicio	URL o conexión
Frontend	http://localhost:5173
Login administrativo	http://localhost:5173/admin/login

Cliente totem	http://localhost:5173/client/totem
Backend	http://localhost:3000
Health backend	http://localhost:3000/api/health
Prueba de base de datos	http://localhost:3000/api/db-test
PostgreSQL	localhost:5432

13.7 Credenciales de base de datos en Docker

Campo	Valor
Host desde la maquina local	localhost
Host desde el backend Docker	postgres
Puerto	5432
Base de datos	totem
Usuario	totem
Contraseña	totem_dev_password_change_me

Ejemplo de conexión desde la maquina local:

```
psql -h localhost -p 5432 -U totem -d totem
```

Ejemplo de conexión dentro del contenedor:

```
docker compose exec postgres psql -U totem -d totem
```

13.8 Credenciales de aplicación

Rol	Correo	Contraseña	Descripción
SuperAdmin	superadmin@gmail.com	Qqwerty8!	Usuario inicial creado por el seeder de la migración.
SuperAdmin demo	superadmin@totem.test	Admin123*	Usuario incluido en los datos de demostración SQL.
Admin Cochabamba	admin.cbba@totem.test	Admin123*	Usuario administrador de prueba.
Admin La Paz	admin.lpz@totem.test	Admin123*	Usuario administrador de prueba.

13.9 Persistencia de datos

Docker Compose utiliza volúmenes para conservar la información:

Volumen	Montaje	Contenido
postgres_data	/var/lib/postgresql/data	Datos persistentes de PostgreSQL.
backend_uploads	/app/uploads	Archivos subidos al backend cuando se usa almacenamiento local.

Mientras estos volúmenes no sean eliminados, la información se conserva, aunque los contenedores se detengan.

14. Personalización y configuración

Esta sección describe las principales opciones de personalización y configuración del sistema **TOTEM**, incluyendo variables de entorno, parámetros funcionales y opciones administrables desde el panel. La configuración general del sistema se maneja mediante archivos `.env`, variables de entorno en hosting y opciones disponibles dentro del panel administrativo.

14.1 Archivos principales de configuración

Archivo	Uso
<code>.env</code>	Configuración general del sistema cuando se ejecuta Docker Compose.
<code>.env.example</code>	Plantilla de variables para Docker Compose.
<code>backend/.env</code>	Configuración local del backend.
<code>backend/.env.example</code>	Plantilla de variables del backend.
<code>frontend/.env</code>	Configuración local del frontend.
<code>frontend/.env.example</code>	Plantilla de variables del frontend.
<code>docker-compose.yml</code>	Define los servicios del sistema en Docker.
<code>frontend/nginx.conf</code>	Configura Nginx para servir el frontend y redirigir peticiones al backend en Docker.

Para crear los archivos de configuración desde las plantillas:

```
copy .env.example .env
copy backend\.env.example backend\.env
copy frontend\.env.example frontend\.env
```

En Linux o macOS:

```
cp .env.example .env
cp backend\.env.example backend\.env
cp frontend\.env.example frontend\.env
```

14.2 Configuración general del sistema

Estas variables permiten definir el entorno de ejecución, los puertos y la comunicación entre frontend y backend.

Variable	Descripción	Ejemplo
NODE_ENV	Define el entorno de ejecución del backend.	development / production
PORT	Puerto interno del backend.	3000
FRONTEND_URL	URL del frontend permitida por CORS y sesiones.	https://totem-univalle.vercel.app
VITE_APP_NAME	Nombre visible de la aplicación.	TOTEM
VITE_API_URL	URL base del backend usada por el frontend.	https://totem-backend-univalle.onrender.com
VITE_AUTH_ME_TIMEOUT_MS	Tiempo máximo para validar sesión desde el frontend.	12000

Ejemplo para producción:

```
NODE_ENV=production
PORT=3000
FRONTEND_URL=https://totem-univalle.vercel.app
VITE_APP_NAME=TOTEM
VITE_API_URL=https://totem-backend-univalle.onrender.com
VITE_AUTH_ME_TIMEOUT_MS=12000
```

14.3 Configuración de base de datos

El backend utiliza PostgreSQL. En producción, la base de datos esta alojada en Render PostgreSQL.

Variable	Descripción	Ejemplo
PGHOST	Host de PostgreSQL.	Host de Render PostgreSQL.
PGPORT	Puerto de PostgreSQL.	5432
PGDATABASE	Nombre de la base de datos.	totem
PGUSER	Usuario de conexión.	Usuario generado por Render.
PGPASSWORD	Contraseña del usuario.	Contraseña privada.
PGSSL	Activa SSL para conexiones remotas.	true
PGSSLMODE	Modo SSL.	require
PG_CONNECTION_TIMEOUT_MS	Tiempo máximo de conexión.	10000
PG_POOL_MAX	Máximo de conexiones del pool.	5
SEQUELIZE_POOL_MAX	Máximo de conexiones Sequelize.	5
DB_LOGGING	Activa o desactiva logs SQL.	false

Ejemplo para producción:

```
PGHOST=<HOST_RENDER_POSTGRESQL>
PGPORT=5432
PGDATABASE=<NOMBRE_BD>
PGUSER=<USUARIO_BD>
PGPASSWORD=<CONTRASEÑA_BD>
PGSSL=true
PGSSLMODE=require
PG_CONNECTION_TIMEOUT_MS=10000
PG_POOL_MAX=5
SEQUELIZE_POOL_MAX=5
DB_LOGGING=false
```

14.4 Configuración de seguridad y sesiones

Estas variables controlan la seguridad de sesiones y contraseñas.

Variable	Descripción	Ejemplo
SESSION_SECRET	Clave privada para firmar sesiones.	clave_larga_segura
SESSION_COOKIE_NAME	Nombre de la cookie de sesión.	totem.sid

SESSION_MAX_AGE_MS	Duración de las sesión administrativa.	28800000
SESSION_TABLE_NAME	Tabla donde se almacenan sesiones.	user_sessions
PASSWORD_SALT_ROUNDS	Rondas de cifrado bcrypt para contraseñas.	12

14.5 Configuración de correo SMTP

El sistema utiliza SMTP con Brevo para enviar correo de invitación de usuarios y recuperación de contraseña.

Variable	Descripción	Ejemplo
SMTP_HOST	Servidor SMTP de Brevo.	smtp-relay.brevo.com
SMTP_PORT	Puerto SMTP utilizado.	2525
SMTP_SECURE	Indica si usa conexión segura directa.	false
SMTP_USER	Usuario SMTP.	Usuario de Brevo.
SMTP_PASS	Contraseña SMTP.	Clave privada.
SMTP_FROM	Remitente mostrado en correos.	TOTEM <correo@dominio.com>
EMAIL_APP_NAME	Nombre mostrado en los correos.	TOTEM
EMAIL_LOGO_URL	URL pública del logo para correos.	URL del logo

Ejemplo:

```
SMTP_HOST=smtp-relay.brevo.com
SMTP_PORT=2525
SMTP_SECURE=false
SMTP_USER=<USUARIO SMTP BREVO>
SMTP_PASS=<CONTRASEÑA SMTP BREVO>
SMTP_FROM=TOTEM <correo@dominio.com>
EMAIL_APP_NAME=TOTEM
EMAIL_LOGO_URL=<URL PUBLICA LOGO>
```

14.6 Configuración de almacenamiento de archivos

El sistema permite trabajar con almacenamiento local o con Supabase Storage.

Almacenamiento local

```
STORAGE_PROVIDER=local
LOCAL_UPLOAD_DIR=uploads
PUBLIC_FILES_URL=http://localhost:3000/uploads
```

Supabase Storage

```
STORAGE_PROVIDER=supabase
SUPABASE_URL=<URL_SUPABASE>
SUPABASE_SERVICE_ROLE_KEY=<SERVICE_ROLE_KEY>
SUPABASE_STORAGE_BUCKET=totem-contents
PUBLIC_FILES_URL=<URL_PUBLIC_BUCKET>
```

En producción se utiliza Supabase Storage para almacenar imágenes, videos, PDFs e imágenes asociadas a preguntas del modo interactivo.

14.7 Configuración del cliente tótem

Estas variables controlan el comportamiento del cliente tótem, la vinculación del dispositivo, el clima y el modo preguntas.

Variable	Descripción	Ejemplo
TOTEM_CLIENT_ACCESS_TOKEN_TTL_SECONDS	Duración del token de acceso del tótem.	900
TOTEM_CLIENT_REFRESH_TOKEN_TTL_SECONDS	Duración del token de renovación.	157680000
TOTEM_LINK_CODE_TTL_MINUTES	Duración del código de vinculación.	10
TOTEM_WEATHER_CACHE_TTL_SECONDS	Tiempo de cache del clima.	600
TOTEM_WEATHER_REQUEST_TIMEOUT_MS	Tiempo máximo de consulta a Open-Meteo	4500
TOTEM_QUESTION_MODE_IDLE_TIMEOUT_SECONDS	Tiempo sin uso antes de salir del modo preguntas.	8
TOTEM_QUESTION_MODE_REACTIVATION_COOLDOWN_SECONDS	Tiempo de espera antes de reactivar el modo preguntas.	2
TOTEM_QUESTION_SESSION_IDLE_TIMEOUT_SECONDS	Tiempo de inactividad de una sesión de preguntas.	20

Ejemplo:

```
TOTEM_LINK_CODE_TTL_MINUTES=10
TOTEM_WEATHER_CACHE_TTL_SECONDS=600
TOTEM_QUESTION_MODE_IDLE_TIMEOUT_SECONDS=8
TOTEM_QUESTION_SESSION_IDLE_TIMEOUT_SECONDS=20
```

14.8 Configuración del modo preguntas

El modo preguntas utiliza para metros de búsqueda para encontrar respuestas en documento PDF asignados al tótem.

Variable	Descripción	Ejemplo
TOTEM_QA_SEARCH_MIN_TRIGRAM_SCORE	Puntaje mínimo de similitud.	0.18
TOTEM_QA_SEARCH_MIN_COMBINED_SCORE	Puntaje mínimo combinado.	0.12
TOTEM_QA_SEARCH_FTS_WEIGHT	Peso de búsqueda por texto.	0.72
TOTEM_QA_SEARCH_TRIGRAM_WEIGHT	Peso de similitud trigram.	0.28
TOTEM_QA_MIN_QUERY_WORD_COUNT	Mínimo de palabras en la pregunta.	3
TOTEM_QA_MIN_QUERY_CHARACTER_COUNT	Mínimo de caracteres en la pregunta.	6
TOTEM_QA_MIN_WORD_OVERLAP	Mínimo de palabras coincidentes.	2

Estos valores permiten ajustar que tan estricta o flexible será la búsqueda de respuesta en los documentos PDF.

14.9 Personalización desde el panel administrativo

El sistema permite personalizar varias partes desde el panel administrativo, sin modificar archivos de configuración.

Modulo	Personalización disponible
Usuarios	Crear usuarios, asignar roles, estados y campus.
Tótems	Registrar tótems, asignarlos a campus y generar códigos de vinculación.
Contenidos	Registrar imágenes, videos, noticias, publicidades y PDFs.
Asignaciones	Asignar contenidos a uno o varios tótems con prioridad, orden y fechas.

Notificaciones	Crear notificaciones normales o urgentes y definir destinatarios.
PDFs	Gestionar preguntas extraídas y asociar imágenes a cada pregunta.

14.10 Personalización de reglas y constantes

Algunas opciones están definidas como constantes en el frontend. Si se modifican, se debe recompilar la aplicación.

Archivo	Parámetro
frontend/src/constants/content.ts	Tipos de contenidos, estados, tamaños máximos de archivo y paginación.
frontend/src/constants/totemContent.ts	Límites de contenidos por asignación, estados y modos de asignación.
frontend/src/constants/totemClient.ts	Constantes usadas por el cliente tótem.
frontend/src/constants/notification.ts	Opciones y estados de notificaciones.
frontend/src/constants/user.ts	Opciones relacionadas con usuarios.

Ejemplos de valores personalizables por código:

- Tamaño máximo de imagen: 15 MB
- Tamaño máximo de video: 150 MB
- Tamaño máximo de PDF: 25 MB
- Imágenes por asignación: 8
- Videos por asignación: 3
- PDFs por asignación: 3
- Imágenes por pregunta PDF: 4

Si se cambian estos valores en el frontend, también se deben revisar las validaciones equivalentes del backend para mantener consistencia.

14.11 Aplicar cambios de configuración

En entorno local

Después de modificar backend/.env o frontend/.env, se deben reiniciar los servicios:

```
cd backend
npm run dev
```

```
cd frontend
npm run dev
```

En Docker Compose

Después de modificar .env, reconstruir los contenedores:

```
docker compose down
docker compose up -d --build
```

En hosting

En producción, las variables deben actualizarse desde el panel correspondiente:

Servicio	Lugar de configuración
Vercel	Variables de entorno del proyecto frontend.
Render	Variables de entorno del servicio backend.
Supabase	Configuración del bucket y claves de acceso.
Brevo	Credenciales SMTP.

Luego de modificar variables Vercel o Render, se debe realizar un nuevo despliegue para aplicar los cambios.

15. Seguridad

Esta sección describe las principales consideraciones de seguridad aplicadas en el sistema **TOTEM**, así como recomendaciones para proteger el software, los datos, los usuarios administrativos, los dispositivos tótem y los servicios externos utilizados. El sistema protege credenciales, sesiones, tokens de dispositivos, datos almacenados en PostgreSQL, archivos multimedia, documentos PDF y variables de entorno sensibles.

15.1 Autenticación y sesiones

El panel administrativo utiliza autenticación mediante correo y contraseña. Las sesiones son administradas desde el backend y almacenadas en PostgreSQL.

Medida	Descripción
Contraseñas cifradas	Las contraseñas se almacenan usando hash con bcrypt.
Sesiones del servidor	El usuario mantiene una sesión segura administrada por el backend.
Cookies HTTP-only	La cookie de sesión no puede ser accedida directamente desde JavaScript.
Expiración de sesión	La duración de la sesión se configura mediante SESSION_MAX_AGE_MS.
Validación de usuario activo	Las rutas protegidas verifican que el usuario siga activo.
Recuperación con token	Los enlaces de recuperación de contraseña usan tokens con tiempo de expiración.

La política de contraseña del sistema debe cumplir como mínimo:

- Mínimo 8 caracteres.
- Al menos una letra mayúscula.
- Al menos una letra minúscula.
- Al menos un número.
- Al menos un carácter especial.

15.2 Roles y permisos de acceso

El sistema maneja control de acceso por roles para limitar las acciones de cada usuario.

Rol	Permisos principales
SuperAdmin	Tiene acceso completo al sistema, incluyendo gestión de usuarios, campus, tótems, contenidos, asignaciones, notificaciones y dashboard general.
Admin	Tiene acceso limitado a su campus asignado. Puede gestionar información operativa relacionada con su sede.

Recomendaciones:

- Asignar el rol SuperAdmin solo a usuarios responsables de la administración general.
- Utilizar el rol Admin para operación diaria.
- Desactivar usuarios que ya no deban acceder al sistema.
- Revisar periódicamente los usuarios activos.
- No compartir cuentas administrativas entre varias personas.

15.3 Seguridad del cliente tótem

El cliente tótem no utiliza correo y contraseña. Su acceso se realiza mediante un proceso de vinculación con código temporal.

Medida	Descripción
Código de vinculación	Se genera desde el panel administrativo para asociar un dispositivo tótem.
Expiración del código	La duración se define mediante TOTEM_LINK_CODE_TTL_MINUTES.
Código de un solo uso	Una vez utilizado, queda marcado como usado.
Tokens de acceso	El cliente tótem utiliza tokens para consumir endpoints protegidos.
Tokens hasheados	Los tokens del dispositivo se almacenan de forma segura en la base de datos.
Renovación de sesión	La sesión puede revocarse si el tótem se desvincula o se desactiva.

Recomendaciones:

- Generar códigos de vinculación solo cuando se vaya a enlazar un dispositivo.
- No compartir públicamente los códigos de vinculación.
- Desvincular el tótem si el dispositivo se pierde o deja de usarse.
- Mantener el dispositivo físico protegido.
- Restringir el acceso al sistema operativo del tótem.

15.4 Seguridad del backend y API

El backend aplica controles de seguridad para proteger las solicitudes HTTP y las rutas del sistema.

Control	Descripción
CORS controlado	Solo permite peticiones desde el frontend configurado en FRONTEND_URL.
Helmet	Agregar cabeceras HTTP de seguridad.
Rate limiting	Limita intentos repetidos en login y recuperación de contraseña.
Middlewares de autenticación	Protegen rutas privadas del panel administrativo.
Middlewares de autorización	Verifican roles antes de permitir acciones sensibles.
Validación de entrada	Los datos recibidos son validados antes de ser procesados.
Manejo centralizado de errores	Evita respuestas inconsistentes y facilita el control de fallos.

Limites configurados:

Endpoint	Limite
Login	10 intentos cada 15 minutos.
Recuperación de contraseña	5 solicitudes cada 15 minutos.
Reset de contraseña	20 intentos cada 15 minutos.

Recomendaciones:

- Usar HTTPS en producción.
- Mantener correctamente configurada la variable FRONTEND_URL.
- No exponer rutas administrativas sin autenticación.
- No registrar contraseñas, tokens o claves en logs.
- Mantener actualizadas las dependencias del backend.

15.5 Seguridad de la base de datos

La base de datos PostgreSQL almacena información importante del sistema, como usuarios, sesiones, contenidos, asignaciones, notificaciones, tótems y datos del modo preguntas.

Recomendaciones:

- No exponer PostgreSQL directamente a internet.
- Usar contraseñas fuertes para la base de datos.
- Utilizar SSL/TLS para conexiones remotas, como en Render PostgreSQL.
- No ejecutar scripts de prueba sobre una base de datos de producción.
- Realizar respaldos periódicos.
- Restringir permisos del usuario de base de datos según sea necesario.

Variables sensibles relacionadas:

```
PGHOST=
PGPORT=
PGDATABASE=
PGUSER=
PGPASSWORD=
PGSSL=true
PGSSLMODE=require
```

15.6 Seguridad de variables de entorno y secretos

El sistema utiliza variables de entorno para almacenar configuraciones sensibles.

Variable	Riesgo si se expone
SESSION_SECRET	Puede comprometer la seguridad de las sesiones.
PGPASSWORD	Permite acceso a la base de datos.
SMTP_PASS	Permite uso indebido del servicio de correo.
SUPABASE_SERVICE_ROLE_KEY	Permite acceso privilegiado al almacenamiento de archivos.
POSTGRES_PASSWORD	Permite acceso a PostgreSQL en Docker.

Recomendaciones:

- No subir archivos .env al repositorio.
- Usar valores diferentes para desarrollo, pruebas y producción.
- Rotar claves si existe sospecha de exposición.
- Guardar credenciales en las variables de entorno del hosting.
- Entregar credenciales reales únicamente por canales seguros.

15.7 Seguridad de archivos y contenidos

El sistema permite subir imágenes, videos, PDFs, noticias e imágenes asociadas a preguntas.

Recomendaciones:

- Validar el tipo de archivo antes de almacenarlo.
- Limitar el tamaño máximo de archivos.
- Evitar subir archivos ejecutables.
- No cargar documentos confidenciales si serán visibles públicamente en el tótem.
- Revisar permisos del bucket de Supabase Storage.
- Mantener respaldos de archivos importantes.

15.8 Seguridad de comunicación

Recomendaciones:

- Usar HTTPS en producción.
- Usar WSS para Socket.IO cuando el sitio este publicado con HTTPS.
- Configurar firewall para permitir solo puertos necesarios.
- Publicar el frontend y backend detrás de un proxy seguro si se usa un servidor propio.
- Restringir acceso directo a PostgreSQL.
- Mantener FROTEEND_URL alineado con el dominio real del frontend.

15.9 Seguridad en Docker y despliegue

Cuando el sistema se ejecuta con Docker Compose, se debe proteger los servicios, puertos y volúmenes.

Recomendaciones:

- Cambiar contraseñas por defecto del archivo .env.
- No usar SESSION_SECRET de desarrollo en producción.
- No publicar puertos internos innecesarios.

- Proteger los volúmenes postgres_data y backend_uploads.
- Mantener actualizadas las imágenes base.
- Evitar ejecutar docker compose down -v en producción, ya que elimina datos persistentes.

15.10 Respaldo y recuperación

Se recomienda realizar respaldo periódicos de los componentes más importantes del sistema.

Recurso	Ubicación
Base de datos	Render PostgreSQL o volumen postgres_data en Docker.
Archivos subidos	Supabase Storage o carpeta uploads.
Variables de entorno	Copia segura fuera del repositorio.
Certificados o claves	Almacenamiento seguro de credenciales.

Recomendaciones:

- Programar respaldos automáticos de PostgreSQL.
- Probar restauraciones en un ambiente de prueba.
- Guardar respaldos en una ubicación diferente al servidor principal.
- Proteger los respaldos con acceso restringido.

16. Glosario de términos

Esta sección presenta los terminos técnicos y conceptos utilizados en el manual técnico del sistema **TOTEM**, con el objetivo de facilitar su comprensión.

Termino	Definición
TOTEM	Sistema web para administrar y mostrar contenido digital en tótems informativos.
Tótem digital	Dispositivo o pantalla utilizada para mostrar información, contenidos multimedia y modo preguntas.
Cliente tótem	Interfaz publica que se ejecuta en el dispositivo tótem y muestra los contenidos asignados.
Frontend	Parte visual del sistema con la que interactúan los usuarios.

Backend	Parte del sistema encargado de la lógica, seguridad, conexión con base de datos, archivos, sesiones y servicios externos.
API	Conjunto de rutas o servicios que permiten la comunicación entre frontend y el backend.
API REST	Tipo de API que permite realizar operaciones como crear, consultar, actualizar o eliminar información mediante peticiones HTTP.
Node.js	Entorno que permite ejecutar JavaScript en el servidor. Es utilizado en el backend del sistema.
Express.js	Framework de Node.js usado para crear la API del backend.
React	Librería utilizada para construir la interfaz del frontend.
TypeScript	Lenguaje basado en JavaScript que agrega tipado para mejorar la organización y mantenimiento del código.
Vite	Herramienta utilizada para desarrollar y compilar el frontend de forma rápida.
TailwindCSS	Framework de estilos utilizado para diseñar la interfaz del sistema.
PostgreSQL	Sistema de gestión de base de datos utilizado por TOTEM para almacenar la información del sistema.
Render PostgreSQL	Servicio de base de datos en la nube utilizado para alojar PostgreSQL en el entorno desplegado.
Sequelize	ORM utilizado para trabajar con PostgreSQL desde el backend mediante modelos y migraciones.
ORM	Herramienta que permite trabajar con tabla de base de datos como si fueran objetos dentro del código.
Migración	Archivo que define cambios en la estructura de la base de datos, como creación de tablas o columnas.
Seeder	Script utilizado para insertar datos iniciales o de prueba en la base de datos.
Script SQL	Archivo con instrucciones SQL para crear estructuras o insertar datos en la base de datos.
Clave primaria	Campo que identifica de forma única cada registro de una tabla.
Clave foránea	Campo que relaciona una tabla con otra dentro de la base de datos.
Asignación	Relación entre un contenido y un tótem, indicando qué se mostrará, en qué dispositivo y durante qué periodo.
Código de vinculación	Código temporal generado desde el panel administrativo para asociar un dispositivo físico con un tótem registrado.

Sesión	Información temporal que permite mantener autenticado a un usuario o dispositivo dentro del sistema.
Middleware	Función intermedia del backend que procesa solicitudes antes de llegar al controlador, por ejemplo, para autenticar o validar datos.
Endpoint	Ruta específica de una API que permite realizar una acción, como iniciar sesión o consultar contenidos.
Modo preguntas	Funcionalidad del tótem que permite al usuario realizar consultas por voz y recibir respuestas basadas en documentos PDF.
Socket.IO	Tecnología usada para comunicación en tiempo real entre el backend y el cliente tótem.
WebSocket	Protocolo que permite comunicación en tiempo real entre cliente y servidor.
SMTP	Protocolo utilizado para enviar correos electrónicos, como invitaciones y recuperación de contraseña.
Brevo	Servicio SMTP utilizado en TOTEM para el envío de correos.
Supabase Storage	Servicio usado para almacenar archivos como imágenes, videos, PDFs e imágenes asociadas a preguntas.
Open-Meteo API	Servicio externo utilizado para obtener información del clima mostrada en el cliente tótem.
Docker	Plataforma que permite ejecutar aplicaciones dentro de contenedores.
Docker Compose	Herramienta que permite levantar varios contenedores al mismo tiempo, como frontend, backend y base de datos.
Contenedor	Entorno aislado donde se ejecuta una parte del sistema, como el backend o la base de datos.
Imagen Docker	Plantilla usada para crear contenedores.
Volumen Docker	Espacio persistente donde Docker almacena datos, como la base de datos o archivos subidos.
Nginx	Servidor web utilizado para servir el frontend compilado dentro del contenedor Docker.
Hosting	Servicio donde se aloja una aplicación para que pueda ser accedida desde internet.
Vercel	Plataforma utilizada para desplegar el frontend del sistema.
Render	Plataforma utilizada para desplegar el backend y la base de datos PostgreSQL.
Variable de entorno	Valor de configuración usado por el sistema sin escribirlo directamente en el código.

.env	Archivo donde se guardan variables de entorno para configurar el sistema.
CORS	Mecanismo de seguridad que controla qué dominios pueden comunicarse con el backend.
Cookie	Dato almacenado en el navegador para mantener la sesión del usuario.
HTTP-only	Configuración de cookie que impide que JavaScript acceda directamente a ella.
Token	Código utilizado para validar una acción o sesión, como recuperación de contraseña o vinculación de tótem.
Hash	Transformación segura de datos sensibles, como contraseñas o tokens, para evitar almacenarlos en texto plano.
Bcrypt	Algoritmo utilizado para cifrar contraseñas de forma segura.
SSL/TLS	Protocolos de seguridad utilizados para cifrar la comunicación entre servicios.
HTTPS	Versión segura de HTTP, utilizada para proteger la comunicación entre navegador y servidor.
Health	Ruta o prueba utilizada para verificar que un servicio está funcionando correctamente.
Despliegue	Proceso de publicar una aplicación en un servidor o plataforma de hosting.
Repositorio	Espacio donde se almacena el código fuente del proyecto, normalmente en GitHub.
Rama	Línea de trabajo dentro del repositorio utilizada para desarrollar funcionalidades o mantener versiones del proyecto.
Main	Rama final y estable del proyecto, usada para la entrega.
Develop	Rama de integración donde se unieron y probaron las funcionalidades antes de pasar a main.

17. Referencias y recursos adicionales

Esta sección presenta referencias y recursos técnicos utilizados como apoyo para el desarrollo, configuración, despliegue y mantenimiento del sistema **TOTEM**.

17.1 Documentación interna del proyecto

Recurso	Descripción
README.md	Documentación general del sistema TOTEM.
README_BD.md	Documentación de la base de datos y scripts SQL.
README_DOCKER.md	Uso básico de Docker Compose para levantar el sistema.

17.2 Frontend

Recurso	Enlace	Uso en el proyecto
React Documentation	https://react.dev/	Consulta sobre componentes, estados, hooks e interfaces del frontend.
React Reference	https://react.dev/reference/react	Referencia de APIs y hooks de React.
React Router	https://reactrouter.com/	Uso de rutas, navegación y layouts del frontend.
Vite Documentation	https://vite.dev/guide/	Configuración del proyecto frontend, entorno de desarrollo, build y variables de entorno.
TypeScript Documentation	https://www.typescriptlang.org/docs/	Tipado, interfaces, tipos y configuración del frontend.
Tailwind CSS Documentation	https://tailwindcss.com/docs/installation/tailwind-cli	Estilos, clases utilitarias y personalización visual del sistema.
Lucide React	https://lucide.dev/guide/react/	Iconos usados en la interfaz.

17.3 Backend y API

Recurso	Enlace	Uso en el proyecto
Node.js Documentation	https://nodejs.org/docs/latest/api/	Referencia para el entorno de ejecución del backend.
Express.js Documentation	https://expressjs.com/en/starter/installing/	Creación de rutas, middlewares, API REST y manejo de solicitudes HTTP.
Socket.IO Documentation	https://socket.io/docs/v4/	Comunicación en tiempo real entre backend y cliente totem.

Sequelize Documentation	https://sequelize.org/docs/v6/getting-started/	Modelos, relaciones, consultas, migraciones y conexión con PostgreSQL.
Zod	https://zod.dev/	Uso de validaciones en los datos de entrada.
Nodemailer	https://nodemailer.com/	Referencia para el envío de correo por SMTP.

17.4 Base de datos

Recurso	Enlace	Uso en el proyecto
PostgreSQL Documentation	https://www.postgresql.org/docs/	Referencia para consultas SQL, estructura de base de datos, índices, extensiones y administración.
Sequelize ORM	https://sequelize.org/	Apoyo para el manejo de modelos, migraciones, seeders y relaciones entre entidades.
Sequelize Querying	https://sequelize.org/docs/v6/core-concepts/model-querying-basics/	Referencia para consultas desde el backend.
PostgreSQL Full Text Search	https://www.postgresql.org/docs/9.5/textsearch.html	Búsqueda textual sobre las preguntas y respuestas extraídas de PDFs.
PostgreSQL pg_trgm	https://www.postgresql.org/docs/current/pgtrgm.html	Comparación por similitud de texto para encontrar coincidencias aproximadas entre la pregunta del usuario y las preguntas almacenadas.

17.5 Seguridad y autenticación

Recurso	Enlace	Uso
Helmet	https://helmetjs.github.io/	Cabeceras de seguridad HTTP.
Express Session	https://github.com/expressjs/session	Manejo de sesiones del panel administrativo.
connect-pg-simple	https://github.com/voxpelli/node-connect-pg-simple	Almacenamiento de sesiones en PostgreSQL.

bcrypt	https://www.npmjs.com/package/bcrypt	Hash de contraseñas.
express-rate-limit	https://express-rate-limit.mintlify.app/overview	Límite de intentos en endpoint sensibles.

17.6 Archivos y PDFs

Recurso	Enlace	Uso
Multer	https://github.com/expressjs/multer	Recepción de archivos en el backend.
pdf-parse	https://www.npmjs.com/package/pdf-parse	Librería utilizada para extraer texto desde archivos PDF en el backend.

17.7 Servicios externos y APIs

Recurso	Enlace	Uso en el proyecto
Brevo SMTP Documentation	https://help.brevo.com/hc/en-us/articles/10905415650322-Which-SMTP-port-should-I-use-Port-587-465-or-2525	Configuración del envío de correos mediante SMTP, incluyendo el uso del puerto 2525.
Open-Meteo API Documentation	https://open-meteo.com/en/docs	Consulta de datos climáticos utilizados en el cliente totem.
Mediapipe Hand Landmarker Documentation	https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/web_js?hl=es-419	Detección de gestos con cámara para activar el modo preguntas.
Web Speech API – MDN	https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API	Reconocimiento de voz y síntesis de voz en el navegador.
MediaDevices getUserMedia – MDN	https://developer.mozilla.org/es/docs/Web/API/MediaDevices/getUserMedia	Acceso a cámara y micrófono desde el navegador del cliente totem.

17.8 Despliegue y hosting

Recurso	Enlace	Uso en el proyecto
---------	--------	--------------------

Vercel Documentation	https://vercel.com/docs/frameworks/frontend/vite	Despliegue del frontend React/Vite y configuración de variables de entorno.
Render Documentation	https://render.com/docs	Despliegue del backend y administración de PostgreSQL en Render.
Docker Compose Documentation	https://docs.docker.com/compose/	Ejecución del frontend, backend y base de datos mediante contenedores.
Compose File Reference	https://docs.docker.com/reference/compose-file/	Referencia de docker-compose.yml
Docker CLI Compose	https://docs.docker.com/reference/cli/docker/compose/	Comandos docker compose.
Nginx Documentation	https://nginx.org/en/docs/	Servidor usado para servir frontend y redirigir rutas.
Supabase Storage Documentation	https://supabase.com/docs/guides/storage	Configuración del almacenamiento de archivos en Supabase Storage.

17.9 Herramientas de desarrollo

Recurso	Enlace	Uso
npm Docs	https://docs.npmjs.com/	Gestión de dependencias y scripts.
ESLint	https://eslint.org/docs/latest/	Revisión estática de código.
PostCSS	https://postcss.org/docs/	Procesamiento de CSS.
Autoprefixer	https://github.com/postcss/autoprefixer	Compatibilidad CSS entre navegadores.
Git	https://git-scm.com/docs	Control de versiones.

17.10 Herramientas de apoyo

Recurso	Uso en el proyecto
dbdiagram.io	Elaboración del diagrama entidad-relación de la base de datos.
pgAdmin	Visualización, administración y exportación de estructura de PostgreSQL.

GitHub	Control de versiones, ramas, integración de cambios y entrega del proyecto.
Thunder Client	Pruebas de endpoint del backend.
Visual Studio Code	Editor utilizado para el desarrollo y organización del proyecto.

17.11 Foros y soporte comunitario

Comunidad	Enlace	Uso
Stack Overflow	https://stackoverflow.com/questions	Preguntas técnicas generales.
Docker Community	https://forums.docker.com/	Soporte comunitario sobre Docker y Docker Compose.
PostgreSQL Community	https://www.postgresql.org/community/	Comunidad oficial de PostgreSQL.
React Community	https://react.dev/community	Recursos comunitarios de React.

18. Herramientas de Implementación

Esta sección describe las principales herramientas utilizadas para la implementación del sistema **TOTEM**, incluyendo lenguajes de programación, frameworks, librerías, base de datos, APIs externas y herramientas de despliegue. El sistema se desarrolló con una arquitectura web compuesta por frontend, backend, base de datos, almacenamiento de archivos y servicios externos.

18.1 Lenguajes de programación

Lenguaje	Uso del sistema
JavaScript	Desarrollo del backend con Node.js y Express.
TypeScript	Desarrollo del frontend con React, componentes, servicios y tipos.
SQL	Creación, consulta y carga de datos en PostgreSQL.
HTML	Estructura base de la aplicación web.
CSS	Estilos visuales de la interfaz, apoyado por TailwindCSS.

18.2 Frameworks y tecnologías principales

Herramienta	Área	Uso
Node.js	Backend	Entorno de ejecución del servidor.
Express.js	Backend	Creación de la API REST y servicios web.
React	Frontend	Construcción del panel administrativo y cliente tótem.
TypeScript	Frontend	Tipado y mejor organización del código.
Vite	Frontend	Servidor de desarrollo y compilación del frontend.
TailwindCSS	Frontend	Diseño visual y estilos de la interfaz.
React Router	Frontend	Navegación y rutas protegidas.
PostgreSQL	Base de datos	Almacenamiento persistente del sistema.
Sequelize	Backend / BD	ORM para modelos, relaciones, migraciones y consultas.
Socket.IO	Tiempo real	Comunicación en tiempo real entre backend y cliente totem.
Docker	Despliegue	Contenerización del backend, frontend y base de datos.
Docker Compose	Despliegue	Orquestación de los servicios del sistema.
Nginx	Despliegue	Servir el frontend compilado y redirigir rutas al backend en Docker.

18.3 Librerías principales del backend

Librería	Uso
express	Creación de rutas, middlewares y API HTTP.
sequelize	ORM para trabajar con PostgreSQL.
pg	Cliente PostgreSQL para Node.js.
express-session	Manejo de sesiones administrativas.
connect-pg-simple	Almacenamiento de sesiones en PostgreSQL.
bcrypt	Cifrado y validación de contraseñas.
helmet	Cabeceras de seguridad HTTP.
cors	Control de acceso entre frontend y backend.
express-rate-limit	Límite de intentos en login y recuperación de contraseña.
zod	Validación de datos de entrada.
multer	Recepción y manejo de archivos subidos.
nodemailer	Envío de correos mediante SMTP.
pdf-parse	Extracción de texto desde documentos PDF.

socket.io	Comunicación en tiempo real.
@supabase/supabase-js	Integración con Supabase Storage.
dotenv	Carga de variables de entorno.

18.4 Librerías principales del frontend

Librería	Uso
react	Construcción de componente visuales.
react-dom	Renderizado de React en el navegador.
react-router-dom	Gestión de rutas del panel y cliente totem.
socket.io-client	Comunicación en tiempo real con el backend.
lucide-react	Iconos de interfaz.
react-speech-recognition	Reconocimiento de voz para el modo preguntas.
@mediapipe/tasks-vision	Detección de mano/palma para activar el modo preguntas.
tailwindcss	Estilos de la interfaz.
typescript	Tipado estático del frontend.

18.5 APIs y servicios externos

Servicios / API	Uso en TOTEM
SMTP	Envío de correos de invitación y recuperación de contraseña.
Supabase Storage	Almacenamiento de imágenes, videos, PDFs e imágenes asociadas a preguntas.
Open-Meteo API	Consulta de información del clima para el cliente totem.
MediaPipe Tasks Vision	Detección de gestos mediante cámara.
Web Speech API	Reconocimiento de voz y sostenéis de voz en el navegador.
MediaDevices API	Acceso a cámara y micrófono del dispositivo totem.

18.6 Herramientas de desarrollo

Herramienta	Uso
npm	Instalación de dependencias y ejecución de scripts.
Nodemon	Reinicio automático del backend en desarrollo.
Sequelize CLI	Ejecución de migraciones y seeders.
ESLint	Revisión de calidad de código.
TypeScript Compiler	Validación y compilación de TypeScript.

PostCSS	Procesamiento de estilos.
Autoprefixer	Compatibilidad CSS entre navegadores.
Git	Control de versiones del proyecto.
GitHub	Repositorio del código fuente.

18.7 Herramientas de base de datos

Herramienta	Uso
PostgreSQL 16	Motor de base de datos principal.
Sequelize Models	Representación de tablas y relaciones desde el backend.
Sequelize Migrations	Control versionado de cambios en la estructura de la base de datos.
Sequelize Seeders	Creación de datos iniciales, como el usuario SuperAdmin.
pg_trgm	Extensión usada para búsquedas por similitud en textos del modo preguntas.

18.8 Herramientas de despliegue

Herramienta / Plataforma	Uso
Vercel	Hosting del frontend React.
Render	Hosting del backend y base de datos PostgreSQL.
Dockerfile Backend	Construcción de la imagen del backend.
Dockerfile Frontend	Compilación del frontend y configuración con Nginx.
Docker Compose	Ejecución conjunta del frontend, backend y PostgreSQL en local.
Nginx config	Sirve el SPA y actúa como proxy hacia el backend.
Supabase	Almacenamiento externo de archivos.

19. Bibliografía

Frontend

- React Documentation - <https://react.dev/>
- React Reference - <https://react.dev/reference/react>
- React Router - <https://reactrouter.com/>
- Vite Documentation - <https://vite.dev/guide/>

- TypeScript Documentation - <https://www.typescriptlang.org/docs/>
- Tailwind CSS Documentation - <https://tailwindcss.com/docs/installation/tailwind-cli>
- Lucide React - <https://lucide.dev/guide/react/>

Backend y API

- Node.js Documentation - <https://nodejs.org/docs/latest/api/>
- Express.js Documentation - <https://expressjs.com/en/starter/installing/>
- Socket.IO Documentation - <https://socket.io/docs/v4/>
- Sequelize Documentation - <https://sequelize.org/docs/v6/getting-started/>
- Zod - <https://zod.dev/>
- Nodemailer - <https://nodemailer.com/>

Base de Datos

- PostgreSQL Documentation - <https://www.postgresql.org/docs/>
- Sequelize ORM - <https://sequelize.org/>
- Sequelize Querying - <https://sequelize.org/docs/v6/core-concepts/model-querying-basics/>
- PostgreSQL Full Text Search - <https://www.postgresql.org/docs/current/textsearch.html>
- PostgreSQL pg_trgm - <https://www.postgresql.org/docs/current/pgtrgm.html>

Seguridad y Autenticación

- Helmet - <https://helmetjs.github.io/>
- Express Session - <https://github.com/expressjs/session>
- connect-pg-simple - <https://github.com/voxpelli/node-connect-pg-simple>
- bcrypt - <https://www.npmjs.com/package/bcrypt>
- express-rate-limit - <https://express-rate-limit.mintlify.app/overview>

Archivos y PDFs

- Multer - <https://github.com/expressjs/multer>
- pdf-parse - <https://www.npmjs.com/package/pdf-parse>

Servicios Externos y APIs

- Brevo SMTP Documentation - <https://help.brevo.com/hc/en-us/articles/10905415650322-Which-SMTP-port-should-I-use-Port-587-465-or-2525>
- Open-Meteo API Documentation - <https://open-meteo.com/en/docs>
- MediaPipe Hand Landmarker Documentation - https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/web_js?hl=es-419
- Web Speech API - MDN - https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- MediaDevices getUserMedia - MDN - <https://developer.mozilla.org/es/docs/Web/API/MediaDevices/getUserMedia>

Despliegue y Hosting

- Vercel Documentation - <https://vercel.com/docs/frameworks/frontend/vite>
- Render Documentation - <https://render.com/docs>
- Docker Compose Documentation - <https://docs.docker.com/compose/>
- Compose File Reference - <https://docs.docker.com/reference/compose-file/>
- Docker CLI Compose - <https://docs.docker.com/reference/cli/docker/compose/>
- Nginx Documentation - <https://nginx.org/en/docs/>
- Supabase Storage Documentation - <https://supabase.com/docs/guides/storage>

Herramientas de Desarrollo

- npm Docs - <https://docs.npmjs.com/>
- ESLint - <https://eslint.org/docs/latest/>
- PostCSS - <https://postcss.org/docs/>

- Autoprefixer - <https://github.com/postcss/autoprefixer>
- Git - <https://git-scm.com/docs>

Foros y Soporte

- Stack Overflow - <https://stackoverflow.com/questions>
- Docker Community - <https://forums.docker.com/>
- PostgreSQL Community - <https://www.postgresql.org/community/>
- React Community - <https://react.dev/community>